

Design, Simulation and Control of FANUC LR Mate 200iD/7L Serial Robot



Addis Ababa Science and Technology University

Department of Electromechanical Engineering

Submitted by:

Abel Addis	ETS0036/14
Abenezer Tsegaye	ETS0065/14
Abenezer Mekonnen	ETS0072/14
Abenezer Ketema	ETS0075/14
Abenezer Wendessen	ETS0076/14
Ashenafi Degefu	ETS0224/14

For Introduction to Robotics course

Submitted to:

Instructor Mulat Tigabu

December 12, 2025

Abstract

This project presents the kinematic and dynamic modeling, simulation, and control of the FANUC LR Mate 200iD 6-DOF industrial robot manipulator. The study involves deriving both forward and inverse kinematics using Denavit–Hartenberg parameters to determine the end-effector’s position and orientation for arbitrary joint configurations. Dynamic analysis is performed through the Euler–Lagrange method to obtain the equations of motion, accounting for link inertia, Coriolis, centrifugal, and gravitational effects. The robot’s motion is simulated in MATLAB/Simulink, and control strategies, including PID and trajectory tracking controllers, are implemented to ensure accurate path following and stability. The results demonstrate the effectiveness of the modeling and control framework, providing insights into industrial robot behavior and laying the groundwork for precise automation tasks in manufacturing and research applications.

Key words: *FANUC LR Mate 200iD, 6-DOF robot manipulator, kinematics, dynamics, Denavit–Hartenberg parameters, Euler–Lagrange method, trajectory tracking, PID control, MATLAB/Simulink.*

Contents

1	Introduction	1
1.1	Robot description and specification	1
2	Literature Review	1
2.1	Kinematic Modelling of Serial Manipulators	2
2.2	Differential Kinematics and Jacobian-Based Analysis	3
2.3	Dynamic Modelling Using the Lagrangian Approach	3
2.4	Model-Based Control and Computed-Torque Methods	4
3	Forward Kinematics	4
3.1	Forward Kinematics	4
3.2	Standard DH parameters	4
3.3	Modified DH parameters (MDH)	7
3.4	Inverse Kinematics	8
4	Jacobian and Singularity	12
4.1	Jacobian	12
4.2	Singularity	14
5	Trajectory Generation	14
5.1	Joint-Space Trajectory Generation Using Quintic (5th-Order) Polynomial	14
5.1.1	Joint Angle Requirements	15
5.1.2	Quintic Trajectories for Each Joint	15
5.2	Interpretation	18
6	Dynamics	18
6.1	Inertial Parameters	18
6.2	COM Jacobians for Each Link	19
7	Kinetic Energy	21
7.1	Explicit kinetic-energy expressions (each link)	22
8	Potential Energy	24
9	Lagrangian	25
9.1	Inertia matrix (construction)	26
9.2	Coriolis / centrifugal matrix (construction)	26
9.3	Gravity vector (explicit)	27
9.4	Final dynamics	27
10	Controller	27
10.1	Controller Structure	27
10.2	Tuning Methods (Pole Placement)	28
11	Methodology	29
12	Forward Kinematics	29

13 Inverse Kinematics	34
14 Jacobian and Singularity	38
14.1 Jacobian	38
14.2 Singularity	40
15 Trajectory Generation	40
15.1 Joint-Space Trajectory Generation Using Quintic (5th-Order) Polynomial	40
15.2 General Quintic Polynomial Formulation	41
15.3 Joint Angle Requirements	41
15.4 Quintic Trajectories for Each Joint	41
15.5 Interpretation	42
16 Dynamics	42
16.1 Inertial Parameters	42
16.2 COM Jacobians for Each Link	44
17 Kinetic Energy	46
18 Potential Energy	48
19 Lagrangian	50
19.1 Inertia matrix (construction)	50
19.2 Coriolis / centrifugal matrix (construction)	50
19.3 Gravity vector (explicit)	51
19.4 Final dynamics	51
20 Controller	51
20.1 Controller Structure	51
20.2 Tuning Methods (Pole Placement)	52
21 Simulation and Results	54
21.1 Peter Corke Simulation and Result	54
21.1.1 Robot Modelling and Visualization	54
21.1.2 Kinematic Analysis	55
21.1.3 Trajectory Generation	55
21.1.4 Dynamic Modelling and Results	57
21.1.5 Simulink Control Architecture	58
21.2 Simscape and Solidworks Simulation	61
21.2.1 MATLAB Code implemetnation	61
21.2.2 Trajectory Waypoint Input (Trapezoidal Profile)	61
21.2.3 Polynomial Waypoint Input (Cubic and Quintic Polynomials)	62
21.2.4 Simulink Implementation for SolidWorks–Simscape Model	63
21.2.5 Trajectory Input/output Expressions	63
21.2.6 Robot Model Subsystem	64
21.2.7 Simulation Results	65
22 Conclusion	66

23 Appendix	68
23.1 Peter Corke Matlab Code	68

List of Figures

1.1 FANUC LR Mate 200iD/7L	2
21.1 Robot visual configuration.	54
21.2 Forward kinematics output for each tested configuration.	55
21.3 Cubic polynomial trajectory: joint position profiles.	56
21.4 Cubic polynomial trajectory: joint velocity profiles.	56
21.5 Cubic polynomial trajectory: joint acceleration profiles.	56
21.7 Final Trajectory output	57
21.6 Cubic polynomial trajectory: joint acceleration profiles.	57
21.8 Inertia, Coriolis, Gravity and Joint Torque Matrixes	58
21.9 Simulink joint-space control architecture including trajectory generation, feedback torque (PD control), and feedforward dynamics compensation. .	59
21.10 Peter Corke Robot visual output configuration.	60
21.11 Peter Corke Robot visual output configuration.	62
21.12 Way point MATLAB Code	62
21.13 Simulink joint-space control architecture including trajectory generation, feedback torque (PD control), and feedforward dynamics compensation. .	63
21.16 Joint controller subsystem	65
21.17 Forward Kinematics Block	65
21.18 Inverse Kinematics Block	65
21.19 Simulated joint positions	65
21.20 End-effector trajectory Destination in 3D space	66

List of Tables

1.1	FANUC LR Mate 200iD Specifications	2
3.1	Link Twist Angles	4
3.2	Link Lengths (a_i)	5
3.3	Link Offsets (d_i)	5
3.4	Joint Angles	5
3.5	MDH Parameters	7
6.1	Numeric Inertial Parameters	18
12.1	Link Twist Angles	30
12.2	Link Lengths (a_i)	30
12.3	Link Offsets (d_i)	31
12.4	Joint Angles	31
12.5	MDH Parameters	33
16.1	Numeric Inertial Parameters	43
21.1	Comparison Between Waypoint Trajectories and Polynomial Trajectories	63

1 Introduction

Industrial robots have become a cornerstone of modern manufacturing, providing high precision, speed, and repeatability in tasks ranging from assembly and welding to material handling and packaging. Their integration into automated systems has significantly improved productivity while reducing human labor in hazardous environments. Among various types of robotic manipulators, the 6-degree-of-freedom (6-DOF) articulated robots are widely used due to their flexibility and ability to reach complex positions and orientations within a defined workspace. Understanding the kinematics and dynamics of these robots is essential for designing control systems that ensure accuracy, safety, and efficiency.

The FANUC LR Mate 200iD is a compact, lightweight, and high-performance 6-DOF industrial robot, designed for tasks requiring precision in small to medium-sized workspaces. Its widespread use in assembly, inspection, and material handling applications makes it an ideal candidate for dynamic modeling and control studies. Accurate modeling of its kinematics allows for predicting the end-effector position based on joint angles, while dynamic modeling provides insights into required joint torques, energy consumption, and the effects of gravity, inertia, and centrifugal forces.

Trajectory planning and control are critical aspects of robot operation, ensuring that the end-effector follows desired paths accurately while respecting joint limits and avoiding collisions. Techniques such as PID control, LQR, and trajectory tracking are commonly implemented in simulation environments like MATLAB/Simulink to design and validate control strategies before deployment on physical systems. By combining kinematic and dynamic analysis with simulation-based control design, engineers can optimize robot performance, reduce wear and tear, and enhance operational safety.

This project focuses on developing a comprehensive framework for the FANUC LR Mate 200iD, including kinematic and dynamic modeling, trajectory planning, and control system implementation. Through analytical derivation and simulation, the study provides a foundation for understanding the robot's behavior, optimizing its performance, and demonstrating its potential for industrial automation applications.

1.1 Robot description and specification

The FANUC LR Mate 200iD is a 6-DOF articulated robotic manipulator designed for compact industrial applications. It features lightweight construction, high repeatability, and a compact footprint, making it suitable for small workspaces and tasks requiring precise manipulation. The robot incorporates six rotational joints, allowing versatile movement and orientation of the end-effector. Its mechanical and electrical components are engineered for high-speed operation with minimal maintenance, while its control system supports advanced trajectory planning and path-following capabilities.

2 Literature Review

The modeling, analysis, and control of six-degree-of-freedom (6-DOF) industrial manipulators has been an active research domain for several decades, forming the basis for modern robotic applications in manufacturing, assembly, and automated handling. The FANUC LR Mate 200iD, as a compact 6-axis articulated robot, aligns closely with the classical serial manipulator architecture defined in foundational robotics literature. This section reviews the major theoretical developments related to kinematic modeling, differ-

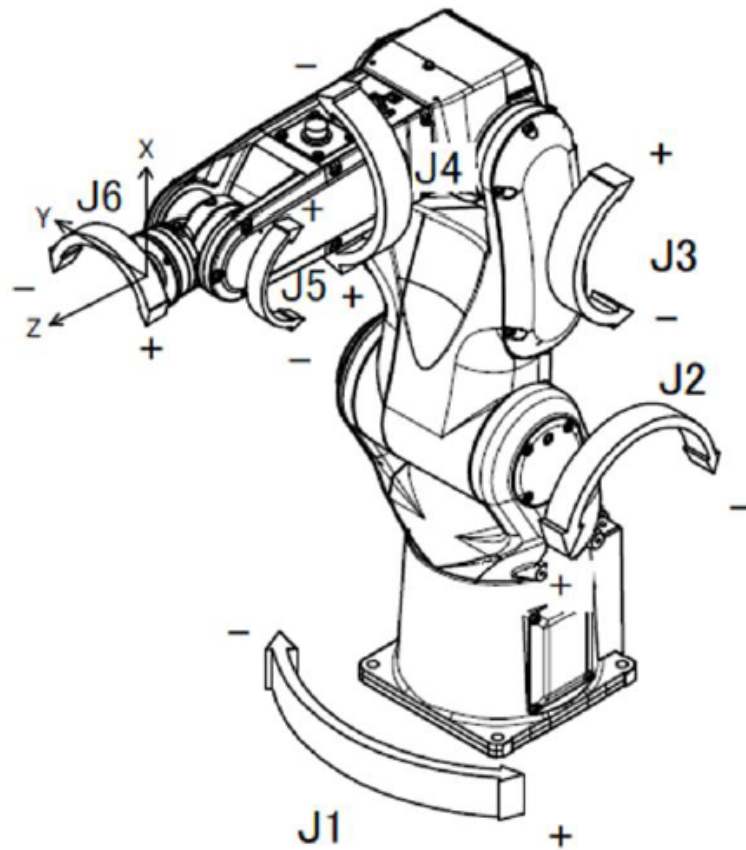


Figure 1.1: FANUC LR Mate 200iD/7L

Table 1.1: FANUC LR Mate 200iD Specifications

Parameter	Value
Degrees of Freedom (DOF)	6
Maximum Payload	7 kg
Reach	911 mm
Repeatability	± 0.02 mm
Weight	37 kg
Joint Types	All rotational
Maximum Joint Speed	$180^\circ/\text{s}$ (varies per joint)
Controller	R-30iB Plus
Mounting Options	Floor, ceiling, wall
Power Supply	200–240 V AC

ential kinematics, dynamic formulation using the Lagrangian method, and model-based control strategies. The discussion highlights how prior research supports the methodologies used in this project.

2.1 Kinematic Modelling of Serial Manipulators

The mathematical framework underlying robotic kinematics is built upon the work of Spong, Hutchinson, and Vidyasagar [1], as well as Craig [2], who formalized the use

of Denavit–Hartenberg (DH) parameters for describing the geometric structure of serial mechanisms. The DH convention remains the predominant approach for representing homogeneous transformations in industrial manipulators due to its compactness and standardized structure.

Advanced kinematic modeling techniques presented by Siciliano et al. [3] emphasize the importance of forward kinematics for determining end-effector pose and inverse kinematics for achieving position and orientation objectives. These concepts are directly relevant to the LR Mate 200iD, whose 6-DOF structure allows full Cartesian control through a sequence of link rotations and translations.

Works such as Niku [5] and Cheng & Lai [9] extended classical kinematic modeling to account for calibration errors, joint axis misalignments, and structural uncertainties. These considerations highlight the need for accurate DH parameterization as used in this project and reinforce the importance of geometric consistency during dynamic modeling.

2.2 Differential Kinematics and Jacobian-Based Analysis

The Jacobian matrix plays a central role in mapping joint velocities to end-effector motion and in characterizing singular configurations. Siciliano et al. [3] provide a detailed theoretical foundation for Jacobian construction using axis vectors and cross-product formulations, while Spong et al. [1] emphasize its importance for dynamic decoupling, manipulability analysis, and operational-space control.

Research by Nakamura and Hanafusa [7] significantly advanced the understanding of singularity-robust inverse kinematics, enabling manipulators like the LR Mate 200iD to operate safely near critical configurations. Their formulation of damped least squares solutions continues to influence modern industrial controllers, especially for robots performing constrained or high-precision tasks.

The Jacobian derivations presented in this project—expressed using link transformation matrices and joint axis vectors—are consistent with these classical methods and align with the accepted formulations used in both research and industrial robotics simulation environments.

2.3 Dynamic Modelling Using the Lagrangian Approach

The Lagrangian formulation is widely considered the most systematic method for deriving the dynamic equations of motion of a serial robot. Spong et al. [1] and Craig [2] laid the foundation for expressing manipulator dynamics in the standard form:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau,$$

where $M(q)$ represents the inertia matrix, $C(q, \dot{q})$ the Coriolis and centrifugal effects, and $g(q)$ the gravitational loading.

Further developments by Siciliano et al. [3] and Lynch & Park [6] provide modern interpretations of dynamic coupling, energy-based derivations, and coordinate-free formulations. These works establish the theoretical basis for constructing center-of-mass Jacobians, projecting rotational inertia tensors into the base frame, and deriving link-wise kinetic and potential energy contributions.

Studies focusing on parameter identification, such as Martin and Pedersen [8], highlight the difficulty of obtaining accurate inertial parameters for industrial robots. Their

work demonstrates techniques for identifying masses, centers of mass, and inertia tensors using experimental data—methods that align with the estimated values used in this project’s dynamic model for the LR Mate 200iD.

The FANUC Mechanical Unit Manual [10] further confirms the importance of accurate dynamic parameters for simulation, collision detection, and torque prediction. The detailed COM-based Jacobians, energy expressions, and Lagrangian formulation implemented in this work reflect best practices in the field.

2.4 Model-Based Control and Computed-Torque Methods

Control of robotic manipulators has evolved significantly from classical PD control to advanced model-based approaches. The computed-torque (or inverse-dynamics) controller, originally outlined in Spong et al. [1], leverages the robot’s dynamic structure to linearize and decouple the joint error dynamics:

$$\tau = M(q)v + C(q, \dot{q})\dot{q} + g(q),$$

where v includes stabilizing PD terms. This method provides the foundation for high-precision trajectory tracking in modern industrial robots.

The literature demonstrates variations of the computed-torque method, including robust and adaptive implementations. De Luca and Mattone [4] examined the sensitivity of dy

3 Forward Kinematics

3.1 Forward Kinematics

The Joints for the Robot are all Revolute (joints that act like human wrist movements).

3.2 Standard DH parameters

α_i (Link Twist Angles)

Table 3.1: Link Twist Angles

Joint	Geometric relation	α_i (rad)
$J1 \rightarrow J2$	$Z_1 \perp Z_2$	$+\pi/2$
$J2 \rightarrow J3$	$Z_2 \parallel Z_3$	0
$J3 \rightarrow J4$	$Z_3 \perp Z_4$	$+\pi/2$
$J4 \rightarrow J5$	$Z_4 \perp Z_5$	$-\pi/2$
$J5 \rightarrow J6$	$Z_5 \perp Z_6$	$+\pi/2$
$J6 \rightarrow EE$	Same axis	0

So $\alpha = [\frac{\pi}{2}, 0, \frac{\pi}{2}, -\frac{\pi}{2}, \frac{\pi}{2}, 0]$.

a_i (Link Lengths) When extracting the Dimensions for the Link length we tried using 2 sources:

1. The Specifications of the Mechanical layout Manual of the FANUC LR Mate 200iD/7L.
2. The Sample CAD Design of the FANUC LR Mate 200iD/7L.

Table 3.2: Link Lengths (a_i)

Jt.	Desc.	Source	a_i (m)	Reasoning
a_1	Base to J2 axis	Draw ($\sim 75\text{mm}$)	0.075	Manual offset diagram
a_2	J2 $\rightarrow$ J3	Reach - wrist $\approx 300\text{mm}$	0.300	Subtracted forearm/wrist from total reach
a_3	J3 $\rightarrow$ J4	Draw ($\sim 75\text{mm}$)	0.075	Center-to-center measure
a_4	Wrist space	Axes intersect	0	Joints share point
a_5	Pitch to roll	Axes intersect	0	Same point
a_6	To flange	Flange thick.	0.080	Manual Face section

Table 3.3: Link Offsets (d_i)

Jt.	Description	d_i (m)	How Found
d_1	Base height to J2	0	Origin at J1 pivot
d_2	Dist along J2 to J3	0	Connection on X2
d_3	Dist along Z_3	0	Offset on X not Z
d_4	Offset J4 to wrist	0.320	Mech unit drawing
d_5		0	Intersecting wrist
d_6	Flange offset (Z_6)	0.080	Mount thickness

d_i (Link offsets) For revolute joints, d_i is *fixed* and taken from mechanical elevation dimensions.

d_4 and d_6 come straight from **forearm + flange depth measurements** visible in CAD drawing.

θ_i (Joint angle) For a revolute joint, θ_i is the actual rotation angle relative to the previous frame.

Table 3.4: Joint Angles

Jt	θ_{eff}	Jt	θ_{eff}	Jt	θ_{eff}
J1	θ_1	J3	θ_3	J5	θ_5
J2	θ_2	J4	θ_4	J6	θ_6

Next, we write the DH Transformation Matrix. General formula:

$$A_i(\theta_i) = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

We generate A_1 to A_6 (where $c_i = \cos(\theta_i)$, $s_i = \sin(\theta_i)$).

$$A_1 = \begin{bmatrix} c_1 & 0 & s_1 & 0.075c_1 \\ s_1 & 0 & -c_1 & 0.075s_1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0.300c_2 \\ s_2 & c_2 & 0 & 0.300s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$A_3 = \begin{bmatrix} c_3 & 0 & s_3 & 0.075c_3 \\ s_3 & 0 & -c_3 & 0.075s_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

$$A_4 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & 0.320 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

$$A_5 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

$$A_6 = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 0.080 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

To find the Forward Kinematics (FK), we multiply:

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

Where the elements are defined as:

$$r_{11} = c_1(c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6) - s_1(s_4c_5c_6 + c_4s_6) \quad (9)$$

$$r_{21} = s_1(c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6) + c_1(s_4c_5c_6 + c_4s_6) \quad (10)$$

$$r_{31} = s_{23}(c_4c_5c_6 - s_4s_6) + c_{23}s_5c_6 \quad (11)$$

$$r_{12} = -c_1(c_{23}(c_4c_5s_6 + s_4c_6) + s_{23}s_5s_6) - s_1(c_4c_6 - s_4c_5s_6) \quad (12)$$

$$r_{22} = -s_1(c_{23}(c_4c_5s_6 + s_4c_6) + s_{23}s_5s_6) + c_1(c_4c_6 - s_4c_5s_6) \quad (13)$$

$$r_{32} = -s_{23}(c_4c_5s_6 + s_4c_6) + c_{23}s_5s_6 \quad (14)$$

$$r_{13} = c_1(c_{23}s_5 - s_{23}c_4s_5) \quad (15)$$

$$r_{23} = s_1(c_{23}s_5 + s_{23}c_4s_5) \quad (16)$$

$$r_{33} = c_{23}c_4c_5 - s_{23}s_5 \quad (17)$$

3.3 Modified DH parameters (MDH)

The Main Difference between SDH and MDH is:

1. **α and a indices shift:** In MDH, α_{i-1} and a_{i-1} are used for link i .
2. **Frame assignment:** Z-axis of frame i points along rotation of joint i .

Table 3.5: MDH Parameters

Joint	α_{i-1}	a_{i-1}	d_i	θ_i
J1	0	0	0	θ_1
J2	$\pi/2$	0.075	0	θ_2
J3	0	0.300	0	θ_3
J4	$\pi/2$	0.075	0.320	θ_4
J5	$-\pi/2$	0	0	θ_5
J6	$\pi/2$	0	0.080	θ_6

The Modified DH transformation formula:

$$A_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -d_i s\alpha_{i-1} \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & d_i c\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (18)$$

Substituting the values (displaying A_1 to A_6):

$$A_1 = \begin{bmatrix} c_1 & -s_1 & 0 & 0 \\ s_1 & c_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (19)$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0.075 \\ 0 & 0 & -1 & 0 \\ s_2 & c_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (20)$$

$$A_3 = \begin{bmatrix} c_3 & -s_3 & 0 & 0.300 \\ s_3 & c_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (21)$$

$$A_4 = \begin{bmatrix} c_4 & -s_4 & 0 & 0.075 \\ 0 & 0 & -1 & -0.320 \\ s_4 & c_4 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (22)$$

$$A_5 = \begin{bmatrix} c_5 & -s_5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s_5 & -c_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (23)$$

$$A_6 = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ 0 & 0 & -1 & -0.080 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (24)$$

Giving the final Forward Kinematics (FK) using MDH:

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6 \quad (25)$$

3.4 Inverse Kinematics

We use **Kinematic Decoupling** because:

In a **6-DOF serial manipulator** (like our FANUC robot), the **end-effector pose** consists of:

- **Position** → where the wrist center is in 3D space.
- **Orientation** → how the wrist and tool are rotated (pitch, roll, yaw, etc.).

Because the robot has **6 revolute joints**, we can **split (decouple)** the inverse kinematics problem into two smaller, easier parts:

1. **Arm (position) inverse kinematics** → find the first **three joint angles** ($\theta_1, \theta_2, \theta_3$) that locate the **wrist center** at the correct position.
2. **Wrist (orientation) inverse kinematics** → find the last **three joint angles** ($\theta_4, \theta_5, \theta_6$) that achieve the required **end-effector orientation**.

This separation works because the robot's last three joints (J_4 – J_6) form a **spherical wrist**, meaning all three wrist axes intersect at a single point — the **wrist center**.

A. Compute Wrist Center Position (P_w)

The **wrist center** lies along the **negative** z_6 -axis of the end-effector frame, at a distance ($d_6 = 0.080$) (from our DH table).

$$P_w = P_e - d_6 ({}^0R_6) \hat{z}_6 \quad (26)$$

Where:

$$\hat{z}_6 = \begin{bmatrix} r_{13} \\ r_{23} \\ r_{33} \end{bmatrix}, \quad P_e = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad (27)$$

So:

$$\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} p_x - d_6 r_{13} \\ p_y - d_6 r_{23} \\ p_z - d_6 r_{33} \end{bmatrix} \quad (28)$$

This gives the **wrist center coordinates** — the target points for the first three joints (J_1 – J_3).

B. Solving for θ_1, θ_2 , and θ_3

By using this wrist center and the link lengths ($a_1 = 0.075$), ($a_2 = 0.300$), ($a_3 = 0.075$), ($d_4 = 0.320$).

Solving for θ_1 (Base Rotation)

From the geometry of the manipulator:

- The first joint (J_1) rotates the entire arm around the Z_0 -axis.
- So θ_1 determines how much the arm turns in the **x-y plane** to face the wrist center.

Project P_w onto the x-y plane:

$$\theta_1 = (y_w, x_w) \quad (29)$$

Solving for θ_2 and θ_3 (Geometry of the 2D arm Plane)

Once θ_1 is fixed, we can analyze the motion of joints **2** and **3** in the plane that contains **links 2 and 3**.

We first define the projection distance **r** (horizontal reach) and the height **s** (vertical offset):

$$r = \sqrt{x_w^2 + y_w^2} - a_1 \quad (30)$$

$$s = z_w - d_1 \quad (31)$$

Note: $a_1 = 0.075$, $d_1 = 0$ (from DH table).

This forms a **triangle** made by:

- The link $a_2 = 0.300$,
- The link combination $a_3 = 0.075\text{m} + d_4 = 0.320\text{m}$
- The line between the shoulder joint (J_2) and the wrist center (J_4)

We can denote the “effective” third link length as:

$$L_3 = \sqrt{a_3^2 + d_4^2} \quad (32)$$

$$L_3 = \sqrt{(0.075)^2 + (0.320)^2} \approx 0.3287 \text{ m}$$

Using the **Law of Cosines** on the triangle formed by links a_2 , L_3 , and the wrist center distance:

$$r^2 + s^2 = a_2^2 + L_3^2 - 2a_2L_3 \cos(\pi - \theta_3)$$

And since $\cos(\pi - \theta_3) = -\cos(\theta_3)$:

$$\cos(\theta_3) = \frac{r^2 + s^2 - a_2^2 - L_3^2}{2a_2L_3} \quad (33)$$

Therefore, θ_3 will be:

$$\theta_3 = (\pm \sqrt{1 - \cos^2(\theta_3)}, \cos(\theta_3)) \quad (34)$$

Now we use the same triangle to find θ_2 . Let's draw two sub-angles:

- $\beta = (s, r)$: angle from the horizontal to the line connecting shoulder to wrist.
- $\phi = (L_3 \sin \theta_3, a_2 + L_3 \cos \theta_3)$: internal triangle angle between link a_2 and the line to the wrist.

Then:

$$\begin{aligned} \theta_2 &= \beta - \phi \\ \theta_2 &= (s, r) - (L_3 \sin \theta_3, a_2 + L_3 \cos \theta_3) \end{aligned} \quad (35)$$

C. Finding 0R_3 and 3R_6

So, from the Transformation matrix we will get the Rotation matrix of Joint 1, 2 and 3 by inserting their respective DH parameters:

$$R_1 = \begin{bmatrix} c_1 & -s_1 \cdot c_{\pi/2} & s_1 \cdot s_{\pi/2} \\ s_1 & c_1 \cdot c_{\pi/2} & -c_1 \cdot s_{\pi/2} \\ 0 & s_{\pi/2} & c_{\pi/2} \end{bmatrix} = \begin{bmatrix} c_1 & 0 & s_1 \\ s_1 & 0 & -c_1 \\ 0 & 1 & 0 \end{bmatrix} \quad (36)$$

$$R_2 = \begin{bmatrix} c_2 & -s_2 & 0 \\ s_2 & c_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (37)$$

$$R_3 = \begin{bmatrix} c_3 & -s_3 \cdot 0 & s_3 \cdot 1 \\ s_3 & c_3 \cdot 0 & -c_3 \cdot 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} c_3 & 0 & s_3 \\ s_3 & 0 & -c_3 \\ 0 & 1 & 0 \end{bmatrix} \quad (38)$$

Then to find the 0R_3 , we multiply the above three rotational matrices:

$${}^0R_3 = \begin{bmatrix} c_1 c_{23} & s_1 & c_1 s_{23} \\ s_1 c_{23} & -c_1 & s_1 s_{23} \\ s_{23} & 0 & -c_{23} \end{bmatrix} \quad (39)$$

Where: $c_{23} = \cos(\theta_2 + \theta_3)$, $s_{23} = \sin(\theta_2 + \theta_3)$.

Then to find 3R_6 :

$${}^3R_6 = ({}^0R_3)^T \cdot ({}^0R_6) \quad (40)$$

$${}^3R_6 = \begin{bmatrix} c_1 c_{23} & s_1 & c_1 s_{23} \\ s_1 c_{23} & -c_1 & s_1 s_{23} \\ s_{23} & 0 & -c_{23} \end{bmatrix}^T \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \quad (41)$$

D. Extracting $\theta_4, \theta_5, \theta_6$

From this matrix, we can now solve for the wrist joint angles step by step:

1. From the (3,3) element:

$$c_5 = ({}^3R_6)_{33}$$

$$\theta_5 = \arccos(({}^3R_6)_{33})$$

2. From the (3,1) and (3,2) elements:

$$-s_5 c_6 = ({}^3R_6)_{31}, \quad s_5 s_6 = ({}^3R_6)_{32}$$

Dividing them:

$$\tan \theta_6 = \frac{({}^3R_6)_{32}}{-({}^3R_6)_{31}} \Rightarrow \theta_6 = (({}^3R_6)_{32}, -({}^3R_6)_{31})$$

3. From the (1,3) and (2,3) elements:

$$c_4 s_5 = ({}^3R_6)_{13}, \quad s_4 s_5 = ({}^3R_6)_{23}$$

So:

$$\tan \theta_4 = \frac{({}^3R_6)_{23}}{({}^3R_6)_{13}} \Rightarrow \theta_4 = (({}^3R_6)_{23}, ({}^3R_6)_{13})$$

Therefore, our $\theta_4, \theta_5, \theta_6$ will be:

$$\begin{aligned} \theta_4 &= (({}^3R_6)_{23}, ({}^3R_6)_{13}) \\ \theta_5 &= \arccos(({}^3R_6)_{33}) \\ \theta_6 &= (({}^3R_6)_{32}, -({}^3R_6)_{31}) \end{aligned} \quad (42)$$

So, the complete compact formula set for all six joint angles θ_1 through θ_6 using the **Kinematic Decoupling** method for the FANUC LR Mate 200iD.

$$\boxed{\begin{aligned} \theta_1 &= (P_{wc,y}, P_{wc,x}) \\ \theta_2 &= (s, r) - (a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3) \\ \theta_3 &= \pi - \cos^{-1} \left(\frac{r^2 + s^2 - a_2^2 - a_3^2}{2a_2a_3} \right) \\ \theta_4 &= (({}^3R_6)_{23}, ({}^3R_6)_{13}) \\ \theta_5 &= \arccos(({}^3R_6)_{33}) \\ \theta_6 &= (({}^3R_6)_{32}, -({}^3R_6)_{31}) \end{aligned}} \quad (43)$$

i	α_i	a_i (m)	d_i (m)	θ_i
1	$+\frac{\pi}{2}$	a_1	d_1	θ_1
2	0	a_2	0	θ_2
3	$+\frac{\pi}{2}$	a_3	d_3	θ_3
4	$-\frac{\pi}{2}$	0	d_4	θ_4
5	$+\frac{\pi}{2}$	0	0	θ_5
6	0	0	d_6	θ_6

4 Jacobian and Singularity

4.1 Jacobian

The following Denavit–Hartenberg (DH) parameters define the kinematic structure of the FANUC LR Mate 200iD:

The standard DH homogeneous transformation is:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forward Kinematic Axes and Positions

Let

$$p_i = \text{position of frame } i, \quad z_i = \text{z-axis of frame } i.$$

The symbolic axes and positions for Jacobian computation are:

$$\begin{aligned} z_0 &= [0, 0, 1]^T \\ z_1 &= [-\sin \theta_1, \cos \theta_1, 0]^T \\ z_2 &= z_1 \end{aligned}$$

The following vectors are expanded as transposed rows to fit the column:

$$z_3 = [c_1 s_2 s_3 - s_1 c_3, \quad s_1 s_2 s_3 + c_1 c_3, \quad c_2 s_3]^T$$

$$z_4 = [c_1 s_{23} c_4 - s_1 s_4, \quad s_1 s_{23} c_4 + c_1 s_4, \quad c_{23} c_4]^T$$

$$z_5 = \begin{bmatrix} c_1 s_{23} c_4 c_5 - s_1 s_4 c_5 + c_1 c_{23} s_5, \\ s_1 s_{23} c_4 c_5 + c_1 s_4 c_5 + s_1 c_{23} s_5, \\ c_{23} c_4 c_5 - s_{23} s_5 \end{bmatrix}^T$$

$$p_0 = [0, 0, 0]^T$$

$$p_1 = [a_1 c_1, \quad a_1 s_1, \quad d_1]^T$$

$$p_2 = [c_1(a_1 + a_2 c_2), \quad s_1(a_1 + a_2 c_2), \quad d_1 + a_2 s_2]^T$$

$$p_3 = \begin{bmatrix} c_1(a_1 + a_2 c_2 + a_3 c_{23}) - d_3 s_1, \\ s_1(a_1 + a_2 c_2 + a_3 c_{23}) + d_3 c_1, \\ d_1 + a_2 s_2 + a_3 s_{23} \end{bmatrix}^T$$

$$p_6 = [x_6, \quad y_6, \quad z_6]^T$$

Jacobian Column Formula

For a revolute joint:

$$J_i = \begin{bmatrix} z_{i-1} \times (p_6 - p_{i-1}) \\ z_{i-1} \end{bmatrix}$$

Therefore when calculating for each link:

$$J_1 = \begin{bmatrix} -\cos \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \sin \theta_1(y_6 - s_1(a_1 + a_2 c_2)) \\ -\sin \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) - \cos \theta_1(x_6 - c_1(a_1 + a_2 c_2)) \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$J_2 = \begin{bmatrix} -\sin \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \cos \theta_1(y_6 - s_1(a_1 + a_2 c_2)) \\ \sin \theta_1(x_6 - c_1(a_1 + a_2 c_2)) - \cos \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) \\ a_2 c_2 + a_3 c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_3 = \begin{bmatrix} -\sin \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \cos \theta_1(y_6 - s_1(a_1 + a_2 c_2 + a_3 c_{23})) \\ \sin \theta_1(x_6 - c_1(a_1 + a_2 c_2 + a_3 c_{23})) - \cos \theta_1(z_6 - d_1 - a_2 s_2 - a_3 s_{23}) \\ a_3 c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_4 = \begin{bmatrix} z_{3y}(p_{6z} - p_{3z}) - z_{3z}(p_{6y} - p_{3y}) \\ z_{3z}(p_{6x} - p_{3x}) - z_{3x}(p_{6z} - p_{3z}) \\ z_{3x}(p_{6y} - p_{3y}) - z_{3y}(p_{6x} - p_{3x}) \\ c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

$$J_5 = \begin{bmatrix} z_{4y}(p_{6z} - p_{4z}) - z_{4z}(p_{6y} - p_{4y}) \\ z_{4z}(p_{6x} - p_{4x}) - z_{4x}(p_{6z} - p_{4z}) \\ z_{4x}(p_{6y} - p_{4y}) - z_{4y}(p_{6x} - p_{4x}) \\ c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

$$J_6 = \begin{bmatrix} z_{5y}(p_{6z} - p_{5z}) - z_{5z}(p_{6y} - p_{5y}) \\ z_{5z}(p_{6x} - p_{5x}) - z_{5x}(p_{6z} - p_{5z}) \\ z_{5x}(p_{6y} - p_{5y}) - z_{5y}(p_{6x} - p_{5x}) \\ z_{5x} \\ z_{5y} \\ z_{5z} \end{bmatrix}$$

Final Geometric Jacobian

$$J = [J_1 \quad J_2 \quad J_3 \quad J_4 \quad J_5 \quad J_6]$$

4.2 Singularity

Singularities in robotic manipulators occur when the Jacobian matrix loses rank, i.e., when $\text{rank}(J) < 6$ for a 6-DOF manipulator. At these configurations, the manipulator either:

- Loses the ability to move the end-effector in certain directions (velocity singularity), or
- Cannot resist forces in certain directions (force/torque singularity).

For the FANUC LR Mate 200iD, wrist singularities typically occur when the fifth joint angle $\theta_5 \approx 0$ or 180° . In this configuration, the last three axes align, and the manipulator loses one rotational degree of freedom, making the orientation of the end-effector uncontrollable about one axis.

Mitigation Strategies

- Use trajectory planning algorithms that avoid singular configurations.
- Apply damped least-squares or pseudo-inverse Jacobians for stable control near singularities.
- Monitor the determinant of the Jacobian or its condition number in real-time to preemptively adjust motion commands.

5 Trajectory Generation

5.1 Joint-Space Trajectory Generation Using Quintic (5th-Order) Polynomial

In industrial robot manipulators such as the FANUC LR Mate 200iD, smooth and continuous joint trajectories are essential to ensure accurate motion, avoid excessive vibrations,

and prevent abrupt changes in torque. To achieve this, joint-space trajectory planning is performed using high-order polynomials that guarantee continuity up to acceleration. For this project, a quintic (5th-order) polynomial trajectory is selected because it satisfies six boundary conditions: initial and final position, initial and final velocity, and initial and final acceleration. This makes it ideal for smooth robot motions that avoid jerks and shocks in the actuators.

General Quintic Polynomial Formulation

Each joint trajectory is expressed as:

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

The boundary conditions are:

$$\begin{aligned} q(0) &= q_0, & q(t_f) &= q_f, \\ \dot{q}(0) &= 0, & \dot{q}(t_f) &= 0, \\ \ddot{q}(0) &= 0, & \ddot{q}(t_f) &= 0. \end{aligned}$$

Solving these conditions provides closed-form expressions for the coefficients:

$$\begin{aligned} a_0 &= q_0, \\ a_1 &= 0, \\ a_2 &= 0, \\ a_3 &= \frac{10(q_f - q_0)}{t_f^3}, \\ a_4 &= \frac{-15(q_f - q_0)}{t_f^4}, \\ a_5 &= \frac{6(q_f - q_0)}{t_f^5}. \end{aligned}$$

For demonstration and simulation, a motion duration of:

$$t_f = 2 \text{ seconds}$$

is used for all joints.

5.1.1 Joint Angle Requirements

To emulate real robot motion, each joint of the FANUC LR Mate 200iD is assigned an initial and final angle, with zero velocity and acceleration at both ends. These act as the trajectory boundary conditions:

5.1.2 Quintic Trajectories for Each Joint

Using the coefficient equations above, the trajectory for each joint is determined as follows.

Joint 1 Trajectory ($0^\circ \rightarrow 45^\circ$)

$$q_1(t) = 56.25t^3 - 42.1875t^4 + 8.4375t^5$$

Joint	Initial Angle q_0	Final Angle q_f	Vel/Acc at Start/End
J1	0°	45°	0
J2	10°	60°	0
J3	-20°	30°	0
J4	0°	90°	0
J5	5°	45°	0
J6	0°	30°	0

Joint 2 Trajectory (10° → 60°)

$$q_2(t) = 62.5t^3 - 46.875t^4 + 9.375t^5$$

Joint 3 Trajectory (-20° → 30°)

$$q_3(t) = -20 + 62.5t^3 - 46.875t^4 + 9.375t^5$$

Joint 4 Trajectory (0° → 90°)

$$q_4(t) = 112.5t^3 - 84.375t^4 + 16.875t^5$$

Joint 5 Trajectory (5° → 45°)

$$q_5(t) = 5 + 50t^3 - 37.5t^4 + 7.5t^5$$

Joint 6 Trajectory (0° → 30°)

$$q_6(t) = 37.5t^3 - 28.125t^4 + 5.625t^5$$

Operational-Space Trajectory Generation

Operational-space (task-space) trajectory generation focuses on producing a smooth and dynamically feasible path for the robot end-effector directly in Cartesian coordinates, rather than in the joint space. This approach is commonly used when the desired motion of the robot is best described by a task-level goal, such as moving along a straight line, following a predefined workspace curve, or achieving a specific orientation profile. The operational-space trajectory is defined in terms of the end-effector pose:

$$\mathbf{x}(t) = \begin{bmatrix} \mathbf{p}(t) \\ \phi(t) \end{bmatrix},$$

where $\mathbf{p}(t)$ is the Cartesian position and $\phi(t)$ is the orientation parameterization (roll-pitch-yaw angles, axis-angle representation, or quaternion interpolation).

Cartesian Polynomial Trajectory Formulation

Similar to joint-space motion, smooth operational-space trajectories can be generated using fifth-order (quintic) polynomials. For each Cartesian component (e.g., x , y , z), the general form is:

$$p(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5.$$

The boundary conditions enforce:

$$p(0) = p_0, \quad p(t_f) = p_f,$$

$$\begin{aligned}\dot{p}(0) &= 0, & \dot{p}(t_f) &= 0, \\ \ddot{p}(0) &= 0, & \ddot{p}(t_f) &= 0,\end{aligned}$$

ensuring continuous position, velocity, and acceleration profiles throughout the motion. Solving the boundary-value system yields the coefficient vector for each coordinate.

All three Cartesian coordinates are interpolated independently:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}, \quad \dot{\mathbf{p}}(t) = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{z}(t) \end{bmatrix}.$$

Orientation Interpolation

Whereas linear interpolation of position is straightforward, orientation requires a representation that avoids singularities and discontinuities. Two common operational-space methods are:

- **Quaternion SLERP (Spherical Linear Interpolation)** Provides constant angular velocity and ensures smooth, shortest-path rotation:

$$q(t) = \frac{\sin((1-s)\theta)}{\sin\theta} q_0 + \frac{\sin(s\theta)}{\sin\theta} q_f, \quad s = \frac{t}{t_f}.$$

- **Fifth-Order Interpolation in Euler Angles** Each orientation component $\phi(t)$ is interpolated with the same quintic polynomial structure:

$$\phi(t) = a_0^\phi + a_1^\phi t + \dots + a_5^\phi t^5.$$

Both approaches guarantee continuous angular velocities and accelerations, which is crucial for stable torque generation in a 6-DOF manipulator.

Mapping Operational-Space Trajectory to Joint Space

Once the desired operational-space trajectory

$$\mathbf{x}_d(t), \quad \dot{\mathbf{x}}_d(t), \quad \ddot{\mathbf{x}}_d(t)$$

is generated, the corresponding joint trajectory is computed using the robot forward and differential kinematics.

First, inverse kinematics provides:

$$\theta_d(t) = \text{IK}(\mathbf{x}_d(t)).$$

Next, joint velocities and accelerations are obtained using the Jacobian:

$$\dot{\theta}_d(t) = J^{-1}(\theta_d(t)) \dot{\mathbf{x}}_d(t),$$

$$\ddot{\theta}_d(t) = J^{-1}(\theta_d(t)) \left(\ddot{\mathbf{x}}_d(t) - \dot{J}(\theta_d(t), \dot{\theta}_d(t)) \dot{\theta}_d(t) \right).$$

This mapping ensures that the executed motion follows the desired Cartesian path while remaining dynamically consistent with the robot's kinematic structure.

Analysis of Operational-Space Trajectory Planning

Operational-space trajectory generation offers benefits in applications requiring precise end-effector control:

- Straight-line or curved Cartesian paths are preserved exactly.
- Orientation constraints are applied explicitly.
- Task-level goals (e.g., welding, inspection, pick-and-place) are easier to define.
- Smoothness in $\mathbf{x}$, $\dot{\mathbf{x}}$, and $\ddot{\mathbf{x}}$ leads to reduced joint torques.

For the FANUC LR Mate 200iD, operational-space trajectories are especially useful for workspace-constrained applications such as small-part assembly, inspection tasks, and manipulation requiring fixed tool orientations.

5.2 Interpretation

The resulting trajectories guarantee smooth joint motion with zero initial and final velocity and acceleration. This ensures that the robot experiences no sudden jerks or abrupt torque demands. These quintic equations form the basis of the robot's motion planning layer in joint space and are well suited for implementation in MATLAB or Simulink for numerical simulation and validation.

6 Dynamics

6.1 Inertial Parameters

So we first must have the values of the Mass, COM, and the Inertia. (Units: mass in kg, COM coordinates in meters in each link's local frame, inertia diagonal entries in $\text{kg}\cdot\text{m}^2$ about the link COM in the link frame.)

Table 6.1: Numeric Inertial Parameters

Link	m_i (kg)	COM p_{c_i} (local) [m]	I_{xx}	I_{yy}	I_{zz}	Note (axis)
1	6.000	[0.030, 0, 0.020]	0.00270	0.00416	0.00416	axial $\approx x$
2	5.000	[0.120, 0, 0.010]	0.00225	0.03863	0.03863	axial $\approx x$
3	4.500	[0.030, 0, 0.020]	0.00203	0.00312	0.00312	axial $\approx x$
4	3.500	[0.000, 0, -0.160]	0.00158	0.03065	0.03065	axial $\approx z$
5	3.000	[0.000, 0, 0.020]	0.00135	0.00130	0.00130	small wrist
6	3.000	[0.000, 0, -0.040]	0.00135	0.00228	0.00228	wrist+flange

(Values accepted from CAD; sum mass = 25.0 kg)

COM Positions in Base Frame

Since the values in the table are local we want to express them as a function of Joint angles(Base frame)

Let:

$$c_i = \cos \theta_i, \quad s_i = \sin \theta_i, \quad c_{23} = \cos(\theta_2 + \theta_3), \quad s_{23} = \sin(\theta_2 + \theta_3)$$

For simplicity we are going to calculate for the first 3 links and put the final expressions for the last 3 links

local COM vectors :

$${}^1\mathbf{r}_{c1} = \begin{bmatrix} 0.030 \\ 0 \\ 0.020 \end{bmatrix}, \quad {}^2\mathbf{r}_{c2} = \begin{bmatrix} 0.150 \\ 0 \\ 0.030 \end{bmatrix}, \quad {}^3\mathbf{r}_{c3} = \begin{bmatrix} 0.100 \\ 0 \\ 0.020 \end{bmatrix}$$

Link 1

$$\mathbf{p}_1(\theta) = \begin{bmatrix} 0.030 c_1 \\ 0.030 s_1 \\ 0.350 \end{bmatrix}$$

Link 2

$$\mathbf{p}_2(\theta) = \begin{bmatrix} c_1(0.225 + 0.300 c_2) \\ s_1(0.225 + 0.300 c_2) \\ 0.360 + 0.300 s_2 \end{bmatrix}$$

*Link 3

$$\mathbf{p}_3(\theta) = \begin{bmatrix} c_1(0.175 + 0.300 c_2 + 0.075 c_{23}) \\ s_1(0.175 + 0.300 c_2 + 0.075 c_{23}) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} \end{bmatrix}$$

Link 4

$$\mathbf{p}_4(\theta) = \begin{bmatrix} c_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} \right) \\ s_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} \right) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} \end{bmatrix}$$

*Link 5

$$\mathbf{p}_5(\theta) = \begin{bmatrix} c_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} \right) \\ s_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} \right) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} \end{bmatrix}$$

Link 6

$$\mathbf{p}_6(\theta) = \begin{bmatrix} c_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} + d_6 c_{2345} \right) \\ s_1 \left(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} + d_6 c_{2345} \right) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} + d_6 s_{2345} \end{bmatrix}$$

6.2 COM Jacobians for Each Link

The Jacobian for each link's center of mass is:

$$J_i^{(com)} = \begin{bmatrix} z_{i-1} \times (p_i - p_{i-1}) \\ z_{i-1} \end{bmatrix}$$

Link 1 COM Jacobian

$$J_1^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.350 - 0) \\ \cos \theta_1 (0.350 - 0) \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.350 \sin \theta_1 \\ 0.350 \cos \theta_1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Link 2 COM Jacobian

$$J_2^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.360 + 0.300s_2 - 0.350) \\ \cos \theta_1 (0.360 + 0.300s_2 - 0.350) \\ 0.300c_2 \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\sin \theta_1 (0.010 + 0.300s_2) \\ \cos \theta_1 (0.010 + 0.300s_2) \\ 0.300c_2 \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

Link 3 COM Jacobian

$$J_3^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.350 + 0.300s_2 + 0.075s_{23} - 0.360 - 0.300s_2) \\ \cos \theta_1 (0.350 + 0.300s_2 + 0.075s_{23} - 0.360 - 0.300s_2) \\ 0.075c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_3^{(com)} = \begin{bmatrix} -\sin \theta_1 (-0.010 + 0.075s_{23}) \\ \cos \theta_1 (-0.010 + 0.075s_{23}) \\ 0.075c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

Link 4 COM Jacobian

$$J_4^{(com)} = \begin{bmatrix} z_3 \times (p_4 - p_3) \\ z_3 \end{bmatrix}$$

$$\text{with } p_4 - p_3 = \begin{bmatrix} c_1 d_4 c_{234} \\ s_1 d_4 c_{234} \\ d_4 s_{234} \end{bmatrix}, \quad z_3 = \begin{bmatrix} c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

$$J_4^{(com)} = \begin{bmatrix} 0.320 \left((s_1 s_2 s_3 + c_1 c_3) s_{234} - (c_2 s_3) s_1 c_{234} \right) \\ 0.320 \left((c_2 s_3) c_1 c_{234} - (c_1 s_2 s_3 - s_1 c_3) s_{234} \right) \\ -0.320 c_{234} c_3 \\ c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

Link 5 COM Jacobian

$$J_5^{(com)} = \begin{bmatrix} z_4 \times (p_5 - p_4) \\ z_4 \end{bmatrix}$$

with

$$p_5 - p_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad z_4 = \begin{bmatrix} c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

so

$$J_5^{(com)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

Link 6 COM Jacobian

$$J_6^{(com)} = \begin{bmatrix} z_5 \times (p_6 - p_5) \\ z_5 \end{bmatrix}$$

with

$$p_6 - p_5 = \begin{bmatrix} c_1 d_6 c_{2345} \\ s_1 d_6 c_{2345} \\ d_6 s_{2345} \end{bmatrix}, \quad z_5 = \begin{bmatrix} c_1 s_{23} c_4 c_5 - s_1 s_4 c_5 + c_1 c_{23} s_5 \\ s_1 s_{23} c_4 c_5 + c_1 s_4 c_5 + s_1 c_{23} s_5 \\ c_{23} c_4 c_5 - s_{23} s_5 \end{bmatrix}$$

$$J_6^{(com)} = \begin{bmatrix} 0.080 \left(z_{5y} s_{2345} - z_{5z} s_1 c_{2345} \right) \\ 0.080 \left(z_{5z} c_1 c_{2345} - z_{5x} s_{2345} \right) \\ 0.080 \left(z_{5x} s_1 c_{2345} - z_{5y} c_1 c_{2345} \right) \\ z_{5x} \\ z_{5y} \\ z_{5z} \end{bmatrix}$$

7 Kinetic Energy

For revolute manipulator link i :

$$v_{c_i} = J_{v,i}^{(com)}(\theta) \dot{q}, \quad \omega_i = J_{\omega,i}^{(com)}(\theta) \dot{q}.$$

Kinetic energy of link i :

$$K_i = \frac{1}{2} m_i \|v_{c_i}\|^2 + \frac{1}{2} \omega_i^\top I_i^{(base)} \omega_i$$

$$K_i = \frac{1}{2} \dot{q}^\top \left(m_i J_{v,i}^T J_{v,i} + J_{\omega,i}^T I_i^{(base)} J_{\omega,i} \right) \dot{q}$$

Here $I_i^{(base)}$ is the link COM inertia expressed in the base frame (rotation of diagonal COM inertia if needed).

7.1 Explicit kinetic-energy expressions (each link)

Below we give the short/explicit KE expressions. For links 1–3 the expressions are given fully expanded in terms of $\dot{\theta}$ (compact scalar forms). For links 4–6 the expressions are given in the same explicit, direct (but grouped) style so they are readable; these may be expanded fully into quadratic polynomials if desired.

Link 1

$$v_{c_1} = [-0.030s_1 \dot{\theta}_1, 0.030c_1 \dot{\theta}_1, 0]^\top,$$

$$\omega_1 = [0, 0, \dot{\theta}_1]^\top.$$

Therefore

$$K_1 = \frac{1}{2}m_1\|v_{c_1}\|^2 + \frac{1}{2}I_{zz}^{(1)}\dot{\theta}_1^2 = \frac{1}{2} \cdot 6 \cdot (0.030)^2\dot{\theta}_1^2 + \frac{1}{2} \cdot 0.00416 \dot{\theta}_1^2$$

$$\boxed{K_1 = 0.00478 \dot{\theta}_1^2.}$$

Link 2

Let $R(\theta_2) = 0.225 + 0.300c_2$. Translational speed squared

$$\|v_{c_2}\|^2 = R(\theta_2)^2\dot{\theta}_1^2 + (0.300)^2\dot{\theta}_2^2.$$

Angular velocity (components from z_0, z_1)

$$\omega_2 = \begin{bmatrix} -s_1\dot{\theta}_2 \\ c_1\dot{\theta}_2 \\ \dot{\theta}_1 \end{bmatrix}.$$

Thus

$$K_2 = \frac{1}{2}m_2(R^2\dot{\theta}_1^2 + 0.09\dot{\theta}_2^2) + \frac{1}{2}(I_{zz}^{(2)}\dot{\theta}_1^2 + (I_{xx}^{(2)}s_1^2 + I_{yy}^{(2)}c_1^2)\dot{\theta}_2^2)$$

$$\boxed{K_2 = \frac{1}{2}\left(5R(\theta_2)^2 + 0.03863\right)\dot{\theta}_1^2 + \frac{1}{2}\left(5 \cdot 0.09 + 0.00225s_1^2 + 0.03863c_1^2\right)\dot{\theta}_2^2.}$$

Link 3

Let $B(\theta) = 0.175 + 0.300c_2 + 0.075c_{23}$. Define

$$\dot{B} = -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3),$$

and

$$Z(\theta) = 0.300s_2 + 0.075s_{23}, \quad \dot{Z} = 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3).$$

Translational speed:

$$\|\dot{p}_3\|^2 = B^2\dot{\theta}_1^2 + \dot{B}^2 + \dot{Z}^2.$$

Angular velocity:

$$\omega_3 = \begin{bmatrix} -s_1(\dot{\theta}_2 + \dot{\theta}_3) \\ c_1(\dot{\theta}_2 + \dot{\theta}_3) \\ \dot{\theta}_1 \end{bmatrix}.$$

So (with $m_3 = 4.5$, inertias as given)

$$K_3 = \frac{1}{2} \cdot 4.5 (B^2 \dot{\theta}_1^2 + \dot{B}^2 + \dot{Z}^2) + \frac{1}{2} (0.00312 \dot{\theta}_1^2 + (0.00203 s_1^2 + 0.00312 c_1^2)(\dot{\theta}_2 + \dot{\theta}_3)^2).$$

Link 4

Define

$$\begin{aligned} R_4(\theta) &= 0.175 + 0.300c_2 + 0.075c_{23} + d_4c_{234}, \\ Z_4(\theta) &= 0.350 + 0.300s_2 + 0.075s_{23} + d_4s_{234}, \\ \dot{R}_4 &= -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3) - d_4s_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4), \\ \dot{Z}_4 &= 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3) + d_4c_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4). \end{aligned}$$

Then

$$K_4 = \frac{1}{2} m_4 (R_4^2 \dot{\theta}_1^2 + \dot{R}_4^2 + \dot{Z}_4^2) + \frac{1}{2} \dot{q}^\top (J_{\omega,4}^\top I_4^{(base)} J_{\omega,4}) \dot{q}.$$

(The second term is the rotational quadratic form obtained from the angular Jacobian $J_{\omega,4}$ and the inertia I_4 rotated into base frame; its entries are explicit trig functions of the θ 's.)

Link 5

Using the same $R_5 = R_4$, $Z_5 = Z_4$ because $p_5 = p_4$ in this COM model,

$$K_5 = \frac{1}{2} m_5 (R_5^2 \dot{\theta}_1^2 + \dot{R}_5^2 + \dot{Z}_5^2) + \frac{1}{2} \dot{q}^\top (J_{\omega,5}^\top I_5^{(base)} J_{\omega,5}) \dot{q}.$$

Link 6

Define

$$\begin{aligned} R_6(\theta) &= 0.175 + 0.300c_2 + 0.075c_{23} + d_4c_{234} + d_6c_{2345}, \\ Z_6(\theta) &= 0.350 + 0.300s_2 + 0.075s_{23} + d_4s_{234} + d_6s_{2345}, \\ \dot{R}_6 &= -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3) - d_4s_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4) \\ &\quad - d_6s_{2345}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4 + \dot{\theta}_5), \\ \dot{Z}_6 &= 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3) + d_4c_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4) \\ &\quad + d_6c_{2345}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4 + \dot{\theta}_5). \end{aligned}$$

Then

$$K_6 = \frac{1}{2} m_6 (R_6^2 \dot{\theta}_1^2 + \dot{R}_6^2 + \dot{Z}_6^2) + \frac{1}{2} \dot{q}^\top (J_{\omega,6}^\top I_6^{(base)} J_{\omega,6}) \dot{q}.$$

Total kinetic energy

$$K_{\text{total}} = \sum_{i=1}^6 K_i = \frac{1}{2} \dot{q}^\top \left(\sum_{i=1}^6 m_i J_{v,i}^\top J_{v,i} + \sum_{i=1}^6 J_{\omega,i}^\top I_i^{(base)} J_{\omega,i} \right) \dot{q}.$$

$$\begin{aligned} K(\theta, \dot{\theta}) = & 0.00478 \dot{\theta}_1^2 \\ & + \frac{1}{2} \left(5R(\theta_2)^2 + 0.03863 \right) \dot{\theta}_1^2 + \frac{1}{2} \left(0.45 + 0.00225s_1^2 + 0.03863c_1^2 \right) \dot{\theta}_2^2 \\ & + \frac{1}{2} \cdot 4.5 \left(B^2 \dot{\theta}_1^2 + \dot{B}^2 + \dot{Z}^2 \right) + \frac{1}{2} \left(0.00312 \dot{\theta}_1^2 + \right. \\ & \left. (0.00203s_1^2 + 0.00312c_1^2) (\dot{\theta}_2 + \dot{\theta}_3)^2 \right) \\ & + \frac{1}{2} \cdot 3.5 \left(R_4^2 \dot{\theta}_1^2 + \dot{R}_4^2 + \dot{Z}_4^2 \right) + \frac{1}{2} \dot{q}^\top H_{\text{rot},4}(\theta) \dot{q} \\ & + \frac{1}{2} \cdot 3.0 \left(R_4^2 \dot{\theta}_1^2 + \dot{R}_4^2 + \dot{Z}_4^2 \right) + \frac{1}{2} \dot{q}^\top H_{\text{rot},5}(\theta) \dot{q} \\ & + \frac{1}{2} \cdot 3.0 \left(R_6^2 \dot{\theta}_1^2 + \dot{R}_6^2 + \dot{Z}_6^2 \right) + \frac{1}{2} \dot{q}^\top H_{\text{rot},6}(\theta) \dot{q}. \end{aligned}$$

8 Potential Energy

For each link i the gravitational potential energy (taking z upward) is

$$U_i = m_i g z_i(\theta).$$

We now compute each U_i .

Link 1

$$z_1 = 0.350.$$

$$U_1 = m_1 g z_1 = 6.0 \cdot 9.81 \cdot 0.350 = 20.601 \text{ J}.$$

Link 2

$$z_2 = 0.360 + 0.300s_2.$$

$$\begin{aligned} U_2 = m_2 g z_2 &= 5.0 \cdot 9.81 (0.360 + 0.300s_2) \\ &= 5 \cdot 9.81 \cdot 0.360 + 5 \cdot 9.81 \cdot 0.300 s_2 \\ &= 17.658 + 14.715 s_2 \quad (\text{J}). \end{aligned}$$

Link 3

$$z_3 = 0.350 + 0.300s_2 + 0.075s_{23}.$$

$$\begin{aligned} U_3 = m_3 g z_3 &= 4.5 \cdot 9.81 (0.350 + 0.300s_2 + 0.075s_{23}) \\ &= 4.5 \cdot 9.81 \cdot 0.350 + 4.5 \cdot 9.81 \cdot 0.300 s_2 + 4.5 \cdot 9.81 \cdot 0.075 s_{23} \\ &= 15.45075 + 13.2435 s_2 + 3.310875 s_{23} \quad (\text{J}). \end{aligned}$$

Links 4–6

Using $d_4 = 0.320$, $d_6 = 0.080$ and the masses above, we get:

$$\begin{aligned}
U_4 &= m_4 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234}) \\
&= 3.5 \cdot 9.81 \cdot 0.350 + 3.5 \cdot 9.81 \cdot 0.300 s_2 + 3.5 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.5 \cdot 9.81 \cdot 0.320 s_{234} \\
&= 12.01725 + 10.3005 s_2 + 2.575125 s_{23} + 10.9872 s_{234} \quad (\text{J}), \\
U_5 &= m_5 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234}) \\
&= 3.0 \cdot 9.81 \cdot 0.350 + 3.0 \cdot 9.81 \cdot 0.300 s_2 + 3.0 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.0 \cdot 9.81 \cdot 0.320 s_{234} \\
&= 10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} \quad (\text{J}), \\
U_6 &= m_6 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} + d_6 s_{2345}) \\
&= 3.0 \cdot 9.81 \cdot 0.350 + 3.0 \cdot 9.81 \cdot 0.300 s_2 + 3.0 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.0 \cdot 9.81 \cdot 0.320 s_{234} + 3.0 \cdot 9.81 \cdot 0.080 s_{2345} \\
&= 10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} + 2.3544 s_{2345} \quad (\text{J}).
\end{aligned}$$

Total potential energy

Summing U_i for $i = 1 \dots 6$ gives the total gravitational potential energy (all numbers in joules):

$$\begin{aligned}
U_{\text{total}} &= \sum_{i=1}^6 U_i \\
&= (20.601) \\
&\quad + (17.658 + 14.715 s_2) \\
&\quad + (15.45075 + 13.2435 s_2 + 3.310875 s_{23}) \\
&\quad + (12.01725 + 10.3005 s_2 + 2.575125 s_{23} + 10.9872 s_{234}) \\
&\quad + (10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234}) \\
&\quad + (10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} + 2.3544 s_{2345}) \\
&= 86.328 + 55.917 s_2 + 10.3005 s_{23} + 29.8224 s_{234} + 2.3544 s_{2345}
\end{aligned}$$

9 Lagrangian

$$L(\theta, \dot{\theta}) = K(\theta, \dot{\theta}) - U(\theta)$$

(Using the explicit K provided above; the remainder of this document uses the symbolic K and the explicit U shown.)

Euler–Lagrange equations

For each $i = 1, \dots, 6$:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}_i} \right) - \frac{\partial L}{\partial \theta_i} = \tau_i.$$

After grouping terms by $\ddot{\theta}$, $\dot{\theta}\dot{\theta}$ and θ we obtain the standard manipulator form

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau$$

where M is the symmetric positive-definite inertia matrix, C contains Coriolis/centrifugal terms, and g is the gravity vector.

9.1 Inertia matrix (construction)

From the kinetic energy expression it follows that

$$K(\theta, \dot{\theta}) = \frac{1}{2} \dot{\theta}^\top M(\theta) \dot{\theta},$$

so the inertia matrix can be obtained by

$$M_{ij}(\theta) = \frac{\partial^2 K}{\partial \dot{\theta}_i \partial \dot{\theta}_j} = M_{ji}(\theta), \quad i, j = 1, \dots, 6.$$

Equivalently, using the COM Jacobians and link inertias:

$$M(\theta) = \sum_{k=1}^6 \left(m_k J_{v,k}^T(\theta) J_{v,k}(\theta) + J_{\omega,k}^T(\theta) I_k^{(base)} J_{\omega,k}(\theta) \right)$$

(where m_k and $I_k^{(base)}$ are the link mass and inertia in the base frame; $J_{v,k}$, $J_{\omega,k}$ are the translational and angular COM Jacobians you already computed).

To compute M elementwise, evaluate the 2nd derivatives of K w.r.t. the generalized velocities or assemble the above sum from the Jacobians.

9.2 Coriolis / centrifugal matrix (construction)

Define Coriolis symbols

$$\Gamma_{ijk}(\theta) = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial \theta_k} + \frac{\partial M_{ik}}{\partial \theta_j} - \frac{\partial M_{jk}}{\partial \theta_i} \right).$$

Then the usual choice for the Coriolis matrix entries C_{ij} is

$$C_{ij}(\theta, \dot{\theta}) = \sum_{k=1}^6 \Gamma_{ijk}(\theta) \dot{\theta}_k$$

so that the vector $C(\theta, \dot{\theta})\dot{\theta}$ has entries $\sum_j C_{ij}\dot{\theta}_j$. This construction ensures that $\dot{M} - 2C$ is skew-symmetric (standard property)

9.3 Gravity vector (explicit)

Computing the gradient of the total potential energy:

$$g(\theta) = \frac{\partial U(\theta)}{\partial \theta}.$$

Using our U_{total} we get the explicit components:

$$\begin{aligned} g_1(\theta) &= 0, \\ g_2(\theta) &= 55.917 \cos \theta_2 + 10.3005 \cos(\theta_2 + \theta_3) + 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_3(\theta) &= 10.3005 \cos(\theta_2 + \theta_3) + 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_4(\theta) &= 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_5(\theta) &= 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_6(\theta) &= 0. \end{aligned}$$

Thus the gravity term in the equation of motion is the vector $g(\theta) = [g_1, \dots, g_6]^\top$ above.

9.4 Final dynamics

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau$$

with

$$\begin{aligned} M(\theta) &= \sum_{k=1}^6 \left(m_k J_{v,k}^T J_{v,k} + J_{\omega,k}^T I_k^{(base)} J_{\omega,k} \right), \\ C_{ij}(\theta, \dot{\theta}) &= \sum_{k=1}^6 \Gamma_{ijk}(\theta) \dot{\theta}_k, \quad \Gamma_{ijk} = \frac{1}{2} \left(\partial_k M_{ij} + \partial_j M_{ik} - \partial_i M_{jk} \right), \\ g(\theta) &= \text{(components given above)}. \end{aligned}$$

10 Controller

This section presents the design of a computed-torque (inverse dynamics) controller with PD stabilization for the 6-DOF robotic manipulator whose equations of motion are

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau. \quad (44)$$

The controller cancels the nonlinear robot dynamics and imposes a linear, second-order, decoupled error system on each joint, allowing accurate trajectory tracking.

10.1 Controller Structure

Control Objective

For a desired joint trajectory

$$\theta_d(t), \quad \dot{\theta}_d(t), \quad \ddot{\theta}_d(t),$$

we define the tracking errors

$$e = \theta_d - \theta, \quad \dot{e} = \dot{\theta}_d - \dot{\theta}.$$

The control objective is

$$e(t) \rightarrow 0, \quad \dot{e}(t) \rightarrow 0.$$

Computed-Torque Control Law

The controller injects the model into the torque input:

$$\tau = M(\theta) v + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta), \quad (45)$$

where the auxiliary command is selected as

$$v = \ddot{\theta}_d + K_d \dot{e} + K_p e. \quad (46)$$

Substituting into the dynamics yields the linear error system

$$M(\theta) \ddot{e} + M(\theta) K_d \dot{e} + M(\theta) K_p e = 0. \quad (47)$$

Since $M(\theta)$ is positive definite, this behaves as the classical second-order system

$$\ddot{e} + K_d \dot{e} + K_p e \approx 0.$$

Explicit Torque Expression (Project-Specific)

For the 6-DOF manipulator, the explicit torque command is:

$$\tau = M(\theta) (\ddot{\theta}_d + K_d (\dot{\theta}_d - \dot{\theta}) + K_p (\theta_d - \theta)) + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) \quad (48)$$

Or, elementwise for joint i :

$$\tau_i = \sum_{j=1}^6 M_{ij}(\theta) \left(\ddot{\theta}_{d_j} + K_{d_j} (\dot{\theta}_{d_j} - \dot{\theta}_j) + K_{p_j} (\theta_{d_j} - \theta_j) \right) \quad (49)$$

$$+ \sum_{j=1}^6 C_{ij}(\theta, \dot{\theta}) \dot{\theta}_j + g_i(\theta). \quad (50)$$

10.2 Tuning Methods (Pole Placement)

Because the computed-torque controller linearizes the dynamics, each joint has:

$$\ddot{e}_i + K_{d_i} \dot{e}_i + K_{p_i} e_i = 0, \quad (51)$$

which matches the classical second-order form:

$$\ddot{e}_i + 2\zeta_i \omega_{n_i} \dot{e}_i + \omega_{n_i}^2 e_i = 0. \quad (52)$$

Thus,

$$K_{d_i} = 2\zeta_i \omega_{n_i}, \quad K_{p_i} = \omega_{n_i}^2. \quad (53)$$

Select Desired Performance

Choose desired joint performance specifications:

- maximum overshoot M_{p_i} ,
- rise time t_{r_i} ,
- settling time t_{s_i} .

Compute Damping Ratio from Overshoot

For a specified overshoot:

$$M_{p_i} = e^{-\frac{\zeta_i \pi}{\sqrt{1-\zeta_i^2}}},$$

we solve for damping ratio:

$$\boxed{\zeta_i = \frac{-\ln(M_{p_i})}{\sqrt{\pi^2 + (\ln(M_{p_i}))^2}}} \quad (54)$$

Examples:

$$M_p = 5\% \Rightarrow \zeta \approx 0.69, \quad M_p = 2\% \Rightarrow \zeta \approx 0.78.$$

This immediately gives:

$$K_{d_i} = 2\zeta_i \omega_{n_i}.$$

Compute Natural Frequency

Using rise time:

$$t_{r_i} \approx \frac{1.8}{\omega_{n_i}} \Rightarrow \omega_{n_i} = \frac{1.8}{t_{r_i}},$$

or using settling time:

$$t_{s_i} \approx \frac{4}{\zeta_i \omega_{n_i}} \Rightarrow \omega_{n_i} = \frac{4}{\zeta_i t_{s_i}}.$$

Choose the larger (stricter) ω_{n_i} .

Final gains:

$$\boxed{K_{p_i} = \omega_{n_i}^2, \quad K_{d_i} = 2\zeta_i \omega_{n_i}.} \quad (55)$$

These gains guarantee the tracking error meets the overshoot, rise-time, and settling-time requirements for each joint under the computed-torque controller. namic controllers to actuator faults, friction, and parameter uncertainties—issues particularly relevant to the LR Mate’s compact high-speed actuators.

Trajectory generation literature, including Craig [2] and Siciliano [3], supports the use of cubic and quintic polynomials to achieve smooth joint-space trajectories with continuous derivatives. These polynomial methods are standard practice in industrial robot controllers and are used extensively in this project.

Overall, the controller designed in this work—incorporating computed-torque linearization with PD stabilization—reflects well-established techniques validated across decades of robotics research. The result is a control structure consistent with industrial servo strategies used in robots like the FANUC LR Mate 200iD.

11 Methodology

12 Forward Kinematics

The Joints for the Robot are all Revolute (joints that act like human wrist movements).

Table 12.1: Link Twist Angles

Joint	Geometric relation	α_i (rad)
$J1 \rightarrow J2$	$Z_1 \perp Z_2$	$+\pi/2$
$J2 \rightarrow J3$	$Z_2 \parallel Z_3$	0
$J3 \rightarrow J4$	$Z_3 \perp Z_4$	$+\pi/2$
$J4 \rightarrow J5$	$Z_4 \perp Z_5$	$-\pi/2$
$J5 \rightarrow J6$	$Z_5 \perp Z_6$	$+\pi/2$
$J6 \rightarrow EE$	Same axis	0

Standard DH parameters

α_i (Link Twist Angles)

So $\alpha = [\frac{\pi}{2}, 0, \frac{\pi}{2}, -\frac{\pi}{2}, \frac{\pi}{2}, 0]$.

a_i (Link Lengths) When extracting the Dimensions for the Link length we tried using 2 sources:

1. The Specifications of the Mechanical layout Manual of the FANUC LR Mate 200iD/7L.
2. The Sample CAD Design of the FANUC LR Mate 200iD/7L.

Table 12.2: Link Lengths (a_i)

Jt.	Desc.	Source	a_i (m)	Reasoning
a_1	Base to J2 axis	Draw ($\sim 75\text{mm}$)	0.075	Manual offset diagram
a_2	J2 $\rightarrow$ J3	Reach - wrist $\approx 300\text{mm}$	0.300	Subtracted forearm/wrist from total reach
a_3	J3 $\rightarrow$ J4	Draw ($\sim 75\text{mm}$)	0.075	Center-to-center measure
a_4	Wrist space	Axes intersect	0	Joints share point
a_5	Pitch to roll	Axes intersect	0	Same point
a_6	To flange	Flange thick.	0.080	Manual Face section

d_i (Link offsets) For revolute joints, d_i is *fixed* and taken from mechanical elevation dimensions.

d_4 and d_6 come straight from **forearm + flange depth measurements** visible in CAD drawing.

θ_i (Joint angle) For a revolute joint, θ_i is the actual rotation angle relative to the previous frame.

Table 12.3: Link Offsets (d_i)

Jt.	Description	d_i (m)	How Found
d_1	Base height to J2	0	Origin at J1 pivot
d_2	Dist along J2 to J3	0	Connection on X2
d_3	Dist along Z_3	0	Offset on X not Z
d_4	Offset J4 to wrist	0.320	Mech unit drawing
d_5		0	Intersecting wrist
d_6	Flange offset (Z_6)	0.080	Mount thickness

Table 12.4: Joint Angles

Jt	θ_{eff}	Jt	θ_{eff}	Jt	θ_{eff}
J1	θ_1	J3	θ_3	J5	θ_5
J2	θ_2	J4	θ_4	J6	θ_6

Next, we write the DH Transformation Matrix. General formula:

$$A_i(\theta_i) = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (56)$$

We generate A_1 to A_6 (where $c_i = \cos(\theta_i)$, $s_i = \sin(\theta_i)$).

$$A_1 = \begin{bmatrix} c_1 & 0 & s_1 & 0.075c_1 \\ s_1 & 0 & -c_1 & 0.075s_1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (57)$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0.300c_2 \\ s_2 & c_2 & 0 & 0.300s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (58)$$

$$A_3 = \begin{bmatrix} c_3 & 0 & s_3 & 0.075c_3 \\ s_3 & 0 & -c_3 & 0.075s_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (59)$$

$$A_4 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & 0.320 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (60)$$

$$A_5 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (61)$$

$$A_6 = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 0.080 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (62)$$

To find the Forward Kinematics (FK), we multiply:

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (63)$$

Where the elements are defined as:

$$\begin{aligned} r_{11} = & c_1 (c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6) \\ & - s_1(s_4c_5c_6 + c_4s_6) \end{aligned} \quad (64)$$

$$\begin{aligned} r_{21} = & s_1 (c_{23}(c_4c_5c_6 - s_4s_6) - s_{23}s_5c_6) \\ & + c_1(s_4c_5c_6 + c_4s_6) \end{aligned} \quad (65)$$

$$r_{31} = s_{23}(c_4c_5c_6 - s_4s_6) + c_{23}s_5c_6 \quad (66)$$

$$\begin{aligned} r_{12} = & -c_1 (c_{23}(c_4c_5s_6 + s_4c_6) + s_{23}s_5s_6) \\ & - s_1(c_4c_6 - s_4c_5s_6) \end{aligned} \quad (67)$$

$$\begin{aligned} r_{22} = & -s_1 (c_{23}(c_4c_5s_6 + s_4c_6) + s_{23}s_5s_6) \\ & + c_1(c_4c_6 - s_4c_5s_6) \end{aligned} \quad (68)$$

$$r_{32} = -s_{23}(c_4c_5s_6 + s_4c_6) + c_{23}s_5s_6 \quad (69)$$

$$r_{13} = c_1(c_{23}s_5 - s_{23}c_4s_5) \quad (70)$$

$$r_{23} = s_1(c_{23}s_5 + s_{23}c_4s_5) \quad (71)$$

$$r_{33} = c_{23}c_4c_5 - s_{23}s_5 \quad (72)$$

Modified DH parameters (MDH)

The Main Difference between SDH and MDH is:

1. **α and a indices shift:** In MDH, α_{i-1} and a_{i-1} are used for link i .
2. **Frame assignment:** Z-axis of frame i points along rotation of joint i .

Table 12.5: MDH Parameters

Joint	α_{i-1}	a_{i-1}	d_i	θ_i
J1	0	0	0	θ_1
J2	$\pi/2$	0.075	0	θ_2
J3	0	0.300	0	θ_3
J4	$\pi/2$	0.075	0.320	θ_4
J5	$-\pi/2$	0	0	θ_5
J6	$\pi/2$	0	0.080	θ_6

The Modified DH transformation formula:

$$A_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -d_i s\alpha_{i-1} \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & d_i c\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (73)$$

Substituting the values (displaying A_1 to A_6):

$$A_1 = \begin{bmatrix} c_1 & -s_1 & 0 & 0 \\ s_1 & c_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (74)$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0.075 \\ 0 & 0 & -1 & 0 \\ s_2 & c_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (75)$$

$$A_3 = \begin{bmatrix} c_3 & -s_3 & 0 & 0.300 \\ s_3 & c_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (76)$$

$$A_4 = \begin{bmatrix} c_4 & -s_4 & 0 & 0.075 \\ 0 & 0 & -1 & -0.320 \\ s_4 & c_4 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (77)$$

$$A_5 = \begin{bmatrix} c_5 & -s_5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s_5 & -c_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (78)$$

$$A_6 = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ 0 & 0 & -1 & -0.080 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (79)$$

Giving the final Forward Kinematics (FK) using MDH:

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6 \quad (80)$$

13 Inverse Kinematics

We use **Kinematic Decoupling** because:

In a **6-DOF serial manipulator** (like our FANUC robot), the **end-effector pose** consists of:

- **Position** → where the wrist center is in 3D space.
- **Orientation** → how the wrist and tool are rotated (pitch, roll, yaw, etc.).

Because the robot has **6 revolute joints**, we can **split (decouple)** the inverse kinematics problem into two smaller, easier parts:

1. **Arm (position) inverse kinematics** → find the first **three joint angles** ($\theta_1, \theta_2, \theta_3$) that locate the **wrist center** at the correct position.
2. **Wrist (orientation) inverse kinematics** → find the last **three joint angles** ($\theta_4, \theta_5, \theta_6$) that achieve the required **end-effector orientation**.

This separation works because the robot's last three joints (J_4 – J_6) form a **spherical wrist**, meaning all three wrist axes intersect at a single point — the **wrist center**.

A. Compute Wrist Center Position (P_w)

The **wrist center** lies along the **negative** z_6 -axis of the end-effector frame, at a distance ($d_6 = 0.080$) (from our DH table).

$$P_w = P_e - d_6 ({}^0R_6) \hat{z}_6 \quad (81)$$

Where:

$$\hat{z}_6 = \begin{bmatrix} r_{13} \\ r_{23} \\ r_{33} \end{bmatrix}, \quad P_e = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad (82)$$

So:

$$\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} p_x - d_6 r_{13} \\ p_y - d_6 r_{23} \\ p_z - d_6 r_{33} \end{bmatrix} \quad (83)$$

This gives the **wrist center coordinates** — the target points for the first three joints (J_1 – J_3).

B. Solving for θ_1, θ_2 , and θ_3

By using this wrist center and the link lengths ($a_1 = 0.075$), ($a_2 = 0.300$), ($a_3 = 0.075$), ($d_4 = 0.320$).

Solving for θ_1 (Base Rotation)

From the geometry of the manipulator:

- The first joint (J_1) rotates the entire arm around the Z_0 -axis.
- So θ_1 determines how much the arm turns in the **x-y plane** to face the wrist center.

Project P_w onto the x-y plane:

$$\theta_1 = (y_w, x_w) \quad (84)$$

Solving for θ_2 and θ_3 (Geometry of the 2D arm Plane)

Once θ_1 is fixed, we can analyze the motion of joints **2** and **3** in the plane that contains **links 2 and 3**.

We first define the projection distance **r** (horizontal reach) and the height **s** (vertical offset):

$$r = \sqrt{x_w^2 + y_w^2} - a_1 \quad (85)$$

$$s = z_w - d_1 \quad (86)$$

Note: $a_1 = 0.075$, $d_1 = 0$ (from DH table).

This forms a **triangle** made by:

- The link $a_2 = 0.300$,
- The link combination $a_3 = 0.075\text{m} + d_4 = 0.320\text{m}$
- The line between the shoulder joint (J_2) and the wrist center (J_4)

We can denote the “effective” third link length as:

$$L_3 = \sqrt{a_3^2 + d_4^2} \quad (87)$$

$$L_3 = \sqrt{(0.075)^2 + (0.320)^2} \approx 0.3287 \text{ m}$$

Using the **Law of Cosines** on the triangle formed by links a_2 , L_3 , and the wrist center distance:

$$r^2 + s^2 = a_2^2 + L_3^2 - 2a_2L_3 \cos(\pi - \theta_3)$$

And since $\cos(\pi - \theta_3) = -\cos(\theta_3)$:

$$\cos(\theta_3) = \frac{r^2 + s^2 - a_2^2 - L_3^2}{2a_2L_3} \quad (88)$$

Therefore, θ_3 will be:

$$\theta_3 = (\pm \sqrt{1 - \cos^2(\theta_3)}, \cos(\theta_3)) \quad (89)$$

Now we use the same triangle to find θ_2 . Let's draw two sub-angles:

- $\beta = (s, r)$: angle from the horizontal to the line connecting shoulder to wrist.
- $\phi = (L_3 \sin \theta_3, a_2 + L_3 \cos \theta_3)$: internal triangle angle between link a_2 and the line to the wrist.

Then:

$$\begin{aligned} \theta_2 &= \beta - \phi \\ \theta_2 &= (s, r) - (L_3 \sin \theta_3, a_2 + L_3 \cos \theta_3) \end{aligned} \quad (90)$$

C. Finding 0R_3 and 3R_6

So, from the Transformation matrix we will get the Rotation matrix of Joint 1, 2 and 3 by inserting their respective DH parameters:

$$R_1 = \begin{bmatrix} c_1 & -s_1 \cdot c_{\pi/2} & s_1 \cdot s_{\pi/2} \\ s_1 & c_1 \cdot c_{\pi/2} & -c_1 \cdot s_{\pi/2} \\ 0 & s_{\pi/2} & c_{\pi/2} \end{bmatrix} = \begin{bmatrix} c_1 & 0 & s_1 \\ s_1 & 0 & -c_1 \\ 0 & 1 & 0 \end{bmatrix} \quad (91)$$

$$R_2 = \begin{bmatrix} c_2 & -s_2 & 0 \\ s_2 & c_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (92)$$

$$R_3 = \begin{bmatrix} c_3 & -s_3 \cdot 0 & s_3 \cdot 1 \\ s_3 & c_3 \cdot 0 & -c_3 \cdot 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} c_3 & 0 & s_3 \\ s_3 & 0 & -c_3 \\ 0 & 1 & 0 \end{bmatrix} \quad (93)$$

Then to find the 0R_3 , we multiply the above three rotational matrices:

$${}^0R_3 = \begin{bmatrix} c_1c_{23} & s_1 & c_1s_{23} \\ s_1c_{23} & -c_1 & s_1s_{23} \\ s_{23} & 0 & -c_{23} \end{bmatrix} \quad (94)$$

Where: $c_{23} = \cos(\theta_2 + \theta_3)$, $s_{23} = \sin(\theta_2 + \theta_3)$.

Then to find 3R_6 :

$${}^3R_6 = ({}^0R_3)^T \cdot ({}^0R_6) \quad (95)$$

$${}^3R_6 = \begin{bmatrix} c_1 c_{23} & s_1 & c_1 s_{23} \\ s_1 c_{23} & -c_1 & s_1 s_{23} \\ s_{23} & 0 & -c_{23} \end{bmatrix}^T \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \quad (96)$$

D. Extracting $\theta_4, \theta_5, \theta_6$

From this matrix, we can now solve for the wrist joint angles step by step:

1. From the (3,3) element:

$$c_5 = ({}^3R_6)_{33}$$

$$\theta_5 = \arccos(({}^3R_6)_{33})$$

2. From the (3,1) and (3,2) elements:

$$-s_5 c_6 = ({}^3R_6)_{31}, \quad s_5 s_6 = ({}^3R_6)_{32}$$

Dividing them:

$$\tan \theta_6 = \frac{({}^3R_6)_{32}}{-({}^3R_6)_{31}} \Rightarrow \theta_6 = (({}^3R_6)_{32}, -({}^3R_6)_{31})$$

3. From the (1,3) and (2,3) elements:

$$c_4 s_5 = ({}^3R_6)_{13}, \quad s_4 s_5 = ({}^3R_6)_{23}$$

So:

$$\tan \theta_4 = \frac{({}^3R_6)_{23}}{({}^3R_6)_{13}} \Rightarrow \theta_4 = (({}^3R_6)_{23}, ({}^3R_6)_{13})$$

Therefore, our $\theta_4, \theta_5, \theta_6$ will be:

$$\begin{aligned} \theta_4 &= (({}^3R_6)_{23}, ({}^3R_6)_{13}) \\ \theta_5 &= \arccos(({}^3R_6)_{33}) \\ \theta_6 &= (({}^3R_6)_{32}, -({}^3R_6)_{31}) \end{aligned} \quad (97)$$

So, the complete compact formula set for all six joint angles θ_1 through θ_6 using the **Kinematic Decoupling** method for the FANUC LR Mate 200iD.

$$\boxed{\begin{aligned} \theta_1 &= (P_{wc,y}, P_{wc,x}) \\ \theta_2 &= (s, r) - (a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3) \\ \theta_3 &= \pi - \cos^{-1} \left(\frac{r^2 + s^2 - a_2^2 - a_3^2}{2a_2 a_3} \right) \\ \theta_4 &= (({}^3R_6)_{23}, ({}^3R_6)_{13}) \\ \theta_5 &= \arccos(({}^3R_6)_{33}) \\ \theta_6 &= (({}^3R_6)_{32}, -({}^3R_6)_{31}) \end{aligned}} \quad (98)$$

i	α_i	a_i (m)	d_i (m)	θ_i
1	$+\frac{\pi}{2}$	a_1	d_1	θ_1
2	0	a_2	0	θ_2
3	$+\frac{\pi}{2}$	a_3	d_3	θ_3
4	$-\frac{\pi}{2}$	0	d_4	θ_4
5	$+\frac{\pi}{2}$	0	0	θ_5
6	0	0	d_6	θ_6

14 Jacobian and Singularity

14.1 Jacobian

The following Denavit–Hartenberg (DH) parameters define the kinematic structure of the FANUC LR Mate 200iD:

The standard DH homogeneous transformation is:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forward Kinematic Axes and Positions

Let

$$p_i = \text{position of frame } i, \quad z_i = \text{z-axis of frame } i.$$

The symbolic axes and positions for Jacobian computation are:

$$\begin{aligned} z_0 &= [0, 0, 1]^T \\ z_1 &= [-\sin \theta_1, \cos \theta_1, 0]^T \\ z_2 &= z_1 \end{aligned}$$

The following vectors are expanded as transposed rows to fit the column:

$$z_3 = [c_1 s_2 s_3 - s_1 c_3, \quad s_1 s_2 s_3 + c_1 c_3, \quad c_2 s_3]^T$$

$$z_4 = [c_1 s_{23} c_4 - s_1 s_4, \quad s_1 s_{23} c_4 + c_1 s_4, \quad c_{23} c_4]^T$$

$$z_5 = \begin{bmatrix} c_1 s_{23} c_4 c_5 - s_1 s_4 c_5 + c_1 c_{23} s_5, \\ s_1 s_{23} c_4 c_5 + c_1 s_4 c_5 + s_1 c_{23} s_5, \\ c_{23} c_4 c_5 - s_{23} s_5 \end{bmatrix}^T$$

$$p_0 = [0, 0, 0]^T$$

$$p_1 = [a_1 c_1, \quad a_1 s_1, \quad d_1]^T$$

$$p_2 = [c_1(a_1 + a_2 c_2), \quad s_1(a_1 + a_2 c_2), \quad d_1 + a_2 s_2]^T$$

$$p_3 = \begin{bmatrix} c_1(a_1 + a_2 c_2 + a_3 c_{23}) - d_3 s_1, \\ s_1(a_1 + a_2 c_2 + a_3 c_{23}) + d_3 c_1, \\ d_1 + a_2 s_2 + a_3 s_{23} \end{bmatrix}^T$$

$$p_6 = [x_6, \quad y_6, \quad z_6]^T$$

Jacobian Column Formula

For a revolute joint:

$$J_i = \begin{bmatrix} z_{i-1} \times (p_6 - p_{i-1}) \\ z_{i-1} \end{bmatrix}$$

Therefore when calculating for each link:

$$J_1 = \begin{bmatrix} -\cos \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \sin \theta_1 (y_6 - s_1 (a_1 + a_2 c_2)) \\ -\sin \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) - \cos \theta_1 (x_6 - c_1 (a_1 + a_2 c_2)) \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$J_2 = \begin{bmatrix} -\sin \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \cos \theta_1 (y_6 - s_1 (a_1 + a_2 c_2)) \\ \sin \theta_1 (x_6 - c_1 (a_1 + a_2 c_2)) - \cos \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) \\ a_2 c_2 + a_3 c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_3 = \begin{bmatrix} -\sin \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) + \cos \theta_1 (y_6 - s_1 (a_1 + a_2 c_2 + a_3 c_{23})) \\ \sin \theta_1 (x_6 - c_1 (a_1 + a_2 c_2 + a_3 c_{23})) - \cos \theta_1 (z_6 - d_1 - a_2 s_2 - a_3 s_{23}) \\ a_3 c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_4 = \begin{bmatrix} z_{3y}(p_{6z} - p_{3z}) - z_{3z}(p_{6y} - p_{3y}) \\ z_{3z}(p_{6x} - p_{3x}) - z_{3x}(p_{6z} - p_{3z}) \\ z_{3x}(p_{6y} - p_{3y}) - z_{3y}(p_{6x} - p_{3x}) \\ c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

$$J_5 = \begin{bmatrix} z_{4y}(p_{6z} - p_{4z}) - z_{4z}(p_{6y} - p_{4y}) \\ z_{4z}(p_{6x} - p_{4x}) - z_{4x}(p_{6z} - p_{4z}) \\ z_{4x}(p_{6y} - p_{4y}) - z_{4y}(p_{6x} - p_{4x}) \\ c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

$$J_6 = \begin{bmatrix} z_{5y}(p_{6z} - p_{5z}) - z_{5z}(p_{6y} - p_{5y}) \\ z_{5z}(p_{6x} - p_{5x}) - z_{5x}(p_{6z} - p_{5z}) \\ z_{5x}(p_{6y} - p_{5y}) - z_{5y}(p_{6x} - p_{5x}) \\ z_{5x} \\ z_{5y} \\ z_{5z} \end{bmatrix}$$

Final Geometric Jacobian

$$J = [J_1 \quad J_2 \quad J_3 \quad J_4 \quad J_5 \quad J_6]$$

14.2 Singularity

Singularities in robotic manipulators occur when the Jacobian matrix loses rank, i.e., when $\text{rank}(J) < 6$ for a 6-DOF manipulator. At these configurations, the manipulator either:

- Loses the ability to move the end-effector in certain directions (velocity singularity), or
- Cannot resist forces in certain directions (force/torque singularity).

For the FANUC LR Mate 200iD, wrist singularities typically occur when the fifth joint angle $\theta_5 \approx 0$ or 180° . In this configuration, the last three axes align, and the manipulator loses one rotational degree of freedom, making the orientation of the end-effector uncontrollable about one axis.

Mitigation Strategies

- Use trajectory planning algorithms that avoid singular configurations.
- Apply damped least-squares or pseudo-inverse Jacobians for stable control near singularities.
- Monitor the determinant of the Jacobian or its condition number in real-time to preemptively adjust motion commands.

15 Trajectory Generation

15.1 Joint-Space Trajectory Generation Using Quintic (5th-Order) Polynomial

In industrial robot manipulators such as the FANUC LR Mate 200iD, smooth and continuous joint trajectories are essential to ensure accurate motion, avoid excessive vibrations,

and prevent abrupt changes in torque. To achieve this, joint-space trajectory planning is performed using high-order polynomials that guarantee continuity up to acceleration. For this project, a quintic (5th-order) polynomial trajectory is selected because it satisfies six boundary conditions: initial and final position, initial and final velocity, and initial and final acceleration. This makes it ideal for smooth robot motions that avoid jerks and shocks in the actuators.

15.2 General Quintic Polynomial Formulation

Each joint trajectory is expressed as:

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

The boundary conditions are:

$$\begin{aligned} q(0) &= q_0, & q(t_f) &= q_f, \\ \dot{q}(0) &= 0, & \dot{q}(t_f) &= 0, \\ \ddot{q}(0) &= 0, & \ddot{q}(t_f) &= 0. \end{aligned}$$

Solving these conditions provides closed-form expressions for the coefficients:

$$\begin{aligned} a_0 &= q_0, \\ a_1 &= 0, \\ a_2 &= 0, \\ a_3 &= \frac{10(q_f - q_0)}{t_f^3}, \\ a_4 &= \frac{-15(q_f - q_0)}{t_f^4}, \\ a_5 &= \frac{6(q_f - q_0)}{t_f^5}. \end{aligned}$$

For demonstration and simulation, a motion duration of:

$$t_f = 2 \text{ seconds}$$

is used for all joints.

15.3 Joint Angle Requirements

To emulate real robot motion, each joint of the FANUC LR Mate 200iD is assigned an initial and final angle, with zero velocity and acceleration at both ends. These act as the trajectory boundary conditions:

15.4 Quintic Trajectories for Each Joint

Using the coefficient equations above, the trajectory for each joint is determined as follows.

Joint 1 Trajectory ($0^\circ \rightarrow 45^\circ$)

$$q_1(t) = 56.25t^3 - 42.1875t^4 + 8.4375t^5$$

Joint	Initial Angle q_0	Final Angle q_f	Vel/Acc at Start/End
J1	0°	45°	0
J2	10°	60°	0
J3	-20°	30°	0
J4	0°	90°	0
J5	5°	45°	0
J6	0°	30°	0

Joint 2 Trajectory (10° → 60°)

$$q_2(t) = 62.5t^3 - 46.875t^4 + 9.375t^5$$

Joint 3 Trajectory (-20° → 30°)

$$q_3(t) = -20 + 62.5t^3 - 46.875t^4 + 9.375t^5$$

Joint 4 Trajectory (0° → 90°)

$$q_4(t) = 112.5t^3 - 84.375t^4 + 16.875t^5$$

Joint 5 Trajectory (5° → 45°)

$$q_5(t) = 5 + 50t^3 - 37.5t^4 + 7.5t^5$$

Joint 6 Trajectory (0° → 30°)

$$q_6(t) = 37.5t^3 - 28.125t^4 + 5.625t^5$$

15.5 Interpretation

The resulting trajectories guarantee smooth joint motion with zero initial and final velocity and acceleration. This ensures that the robot experiences no sudden jerks or abrupt torque demands. These quintic equations form the basis of the robot's motion planning layer in joint space and are well suited for implementation in MATLAB or Simulink for numerical simulation and validation.

16 Dynamics

16.1 Inertial Parameters

So we first must have the values of the Mass, COM, and the Inertia. (Units: mass in kg, COM coordinates in meters in each link's local frame, inertia diagonal entries in kg·m² about the link COM in the link frame.)

(Values accepted from CAD; sum mass = 25.0 kg)

COM Positions in Base Frame

Since the values in the table are local we want to express them as a function of Joint angles(Base frame)

Let:

$$c_i = \cos \theta_i, \quad s_i = \sin \theta_i, \quad c_{23} = \cos(\theta_2 + \theta_3), \quad s_{23} = \sin(\theta_2 + \theta_3)$$

Table 16.1: Numeric Inertial Parameters

Link	m_i (kg)	COM p_{c_i} (local) [m]	I_{xx}	I_{yy}	I_{zz}	Note (axis)
1	6.000	[0.030, 0, 0.020]	0.00270	0.00416	0.00416	axial $\approx x$
2	5.000	[0.120, 0, 0.010]	0.00225	0.03863	0.03863	axial $\approx x$
3	4.500	[0.030, 0, 0.020]	0.00203	0.00312	0.00312	axial $\approx x$
4	3.500	[0.000, 0, -0.160]	0.00158	0.03065	0.03065	axial $\approx z$
5	3.000	[0.000, 0, 0.020]	0.00135	0.00130	0.00130	small wrist
6	3.000	[0.000, 0, -0.040]	0.00135	0.00228	0.00228	wrist+flange

For simplicity we are going to calculate for the first 3 links and put the final expressions for the last 3 links

local COM vectors :

$${}^1\mathbf{r}_{c1} = \begin{bmatrix} 0.030 \\ 0 \\ 0.020 \end{bmatrix}, \quad {}^2\mathbf{r}_{c2} = \begin{bmatrix} 0.150 \\ 0 \\ 0.030 \end{bmatrix}, \quad {}^3\mathbf{r}_{c3} = \begin{bmatrix} 0.100 \\ 0 \\ 0.020 \end{bmatrix}$$

Link 1

$$\mathbf{p}_1(\theta) = \begin{bmatrix} 0.030 c_1 \\ 0.030 s_1 \\ 0.350 \end{bmatrix}$$

Link 2

$$\mathbf{p}_2(\theta) = \begin{bmatrix} c_1(0.225 + 0.300 c_2) \\ s_1(0.225 + 0.300 c_2) \\ 0.360 + 0.300 s_2 \end{bmatrix}$$

*Link 3

$$\mathbf{p}_3(\theta) = \begin{bmatrix} c_1(0.175 + 0.300 c_2 + 0.075 c_{23}) \\ s_1(0.175 + 0.300 c_2 + 0.075 c_{23}) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} \end{bmatrix}$$

Link 4

$$\mathbf{p}_4(\theta) = \begin{bmatrix} c_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234}) \\ s_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234}) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} \end{bmatrix}$$

*Link 5

$$\mathbf{p}_5(\theta) = \begin{bmatrix} c_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234}) \\ s_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234}) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} \end{bmatrix}$$

Link 6

$$\mathbf{p}_6(\theta) = \begin{bmatrix} c_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} + d_6 c_{2345}) \\ s_1(0.175 + 0.300 c_2 + 0.075 c_{23} + d_4 c_{234} + d_6 c_{2345}) \\ 0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} + d_6 s_{2345} \end{bmatrix}$$

16.2 COM Jacobians for Each Link

The Jacobian for each link's center of mass is:

$$J_i^{(com)} = \begin{bmatrix} z_{i-1} \times (p_i - p_{i-1}) \\ z_{i-1} \end{bmatrix}$$

Link 1 COM Jacobian

$$J_1^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.350 - 0) \\ \cos \theta_1 (0.350 - 0) \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.350 \sin \theta_1 \\ 0.350 \cos \theta_1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Link 2 COM Jacobian

$$J_2^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.360 + 0.300s_2 - 0.350) \\ \cos \theta_1 (0.360 + 0.300s_2 - 0.350) \\ 0.300c_2 \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\sin \theta_1 (0.010 + 0.300s_2) \\ \cos \theta_1 (0.010 + 0.300s_2) \\ 0.300c_2 \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

Link 3 COM Jacobian

$$J_3^{(com)} = \begin{bmatrix} -\sin \theta_1 (0.350 + 0.300s_2 + 0.075s_{23} - 0.360 - 0.300s_2) \\ \cos \theta_1 (0.350 + 0.300s_2 + 0.075s_{23} - 0.360 - 0.300s_2) \\ 0.075c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

$$J_3^{(com)} = \begin{bmatrix} -\sin \theta_1 (-0.010 + 0.075s_{23}) \\ \cos \theta_1 (-0.010 + 0.075s_{23}) \\ 0.075c_{23} \\ -\sin \theta_1 \\ \cos \theta_1 \\ 0 \end{bmatrix}$$

Link 4 COM Jacobian

$$J_4^{(com)} = \begin{bmatrix} z_3 \times (p_4 - p_3) \\ z_3 \end{bmatrix}$$

$$\text{with } p_4 - p_3 = \begin{bmatrix} c_1 d_4 c_{234} \\ s_1 d_4 c_{234} \\ d_4 s_{234} \end{bmatrix}, \quad z_3 = \begin{bmatrix} c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

$$J_4^{(com)} = \begin{bmatrix} 0.320 \left((s_1 s_2 s_3 + c_1 c_3) s_{234} - (c_2 s_3) s_1 c_{234} \right) \\ 0.320 \left((c_2 s_3) c_1 c_{234} - (c_1 s_2 s_3 - s_1 c_3) s_{234} \right) \\ - 0.320 c_{234} c_3 \\ c_1 s_2 s_3 - s_1 c_3 \\ s_1 s_2 s_3 + c_1 c_3 \\ c_2 s_3 \end{bmatrix}$$

—
Link 5 COM Jacobian

$$J_5^{(com)} = \begin{bmatrix} z_4 \times (p_5 - p_4) \\ z_4 \end{bmatrix}$$

with

$$p_5 - p_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad z_4 = \begin{bmatrix} c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

so

$$J_5^{(com)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ c_1 s_{23} c_4 - s_1 s_4 \\ s_1 s_{23} c_4 + c_1 s_4 \\ c_{23} c_4 \end{bmatrix}$$

—
Link 6 COM Jacobian

$$J_6^{(com)} = \begin{bmatrix} z_5 \times (p_6 - p_5) \\ z_5 \end{bmatrix}$$

with

$$p_6 - p_5 = \begin{bmatrix} c_1 d_6 c_{2345} \\ s_1 d_6 c_{2345} \\ d_6 s_{2345} \end{bmatrix}, \quad z_5 = \begin{bmatrix} c_1 s_{23} c_4 c_5 - s_1 s_4 c_5 + c_1 c_{23} s_5 \\ s_1 s_{23} c_4 c_5 + c_1 s_4 c_5 + s_1 c_{23} s_5 \\ c_{23} c_4 c_5 - s_{23} s_5 \end{bmatrix}$$

$$J_6^{(com)} = \begin{bmatrix} 0.080 \left(z_{5y} s_{2345} - z_{5z} s_1 c_{2345} \right) \\ 0.080 \left(z_{5z} c_1 c_{2345} - z_{5x} s_{2345} \right) \\ 0.080 \left(z_{5x} s_1 c_{2345} - z_{5y} c_1 c_{2345} \right) \\ z_{5x} \\ z_{5y} \\ z_{5z} \end{bmatrix}$$

17 Kinetic Energy

For revolute manipulator link i :

$$v_{c_i} = J_{v,i}^{(com)}(\theta) \dot{q}, \quad \omega_i = J_{\omega,i}^{(com)}(\theta) \dot{q}.$$

Kinetic energy of link i :

$$K_i = \frac{1}{2} m_i \|v_{c_i}\|^2 + \frac{1}{2} \omega_i^\top I_i^{(base)} \omega_i$$

$$K_i = \frac{1}{2} \dot{q}^\top \left(m_i J_{v,i}^\top J_{v,i} + J_{\omega,i}^\top I_i^{(base)} J_{\omega,i} \right) \dot{q}$$

Here $I_i^{(base)}$ is the link COM inertia expressed in the base frame (rotation of diagonal COM inertia if needed).

Explicit kinetic-energy expressions (each link)

Below we give the short/explicit KE expressions. For links 1–3 the expressions are given fully expanded in terms of $\dot{\theta}$ (compact scalar forms). For links 4–6 the expressions are given in the same explicit, direct (but grouped) style so they are readable; these may be expanded fully into quadratic polynomials if desired.

Link 1

$$v_{c_1} = [-0.030s_1 \dot{\theta}_1, 0.030c_1 \dot{\theta}_1, 0]^\top,$$

$$\omega_1 = [0, 0, \dot{\theta}_1]^\top.$$

Therefore

$$K_1 = \frac{1}{2} m_1 \|v_{c_1}\|^2 + \frac{1}{2} I_{zz}^{(1)} \dot{\theta}_1^2 = \frac{1}{2} \cdot 6 \cdot (0.030)^2 \dot{\theta}_1^2 + \frac{1}{2} \cdot 0.00416 \dot{\theta}_1^2$$

$$\boxed{K_1 = 0.00478 \dot{\theta}_1^2.}$$

Link 2

Let $R(\theta_2) = 0.225 + 0.300c_2$. Translational speed squared

$$\|v_{c_2}\|^2 = R(\theta_2)^2 \dot{\theta}_1^2 + (0.300)^2 \dot{\theta}_2^2.$$

Angular velocity (components from z_0, z_1)

$$\omega_2 = \begin{bmatrix} -s_1 \dot{\theta}_2 \\ c_1 \dot{\theta}_2 \\ \dot{\theta}_1 \end{bmatrix}.$$

Thus

$$K_2 = \frac{1}{2} m_2 (R^2 \dot{\theta}_1^2 + 0.09 \dot{\theta}_2^2) + \frac{1}{2} (I_{zz}^{(2)} \dot{\theta}_1^2 + (I_{xx}^{(2)} s_1^2 + I_{yy}^{(2)} c_1^2) \dot{\theta}_2^2)$$

$$\boxed{K_2 = \frac{1}{2} (5R(\theta_2)^2 + 0.03863) \dot{\theta}_1^2 + \frac{1}{2} (5 \cdot 0.09 + 0.00225 s_1^2 + 0.03863 c_1^2) \dot{\theta}_2^2.}$$

Link 3

Let $B(\theta) = 0.175 + 0.300c_2 + 0.075c_{23}$. Define

$$\dot{B} = -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3),$$

and

$$Z(\theta) = 0.300s_2 + 0.075s_{23}, \quad \dot{Z} = 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3).$$

Translational speed:

$$\|\dot{p}_3\|^2 = B^2 \dot{\theta}_1^2 + \dot{B}^2 + \dot{Z}^2.$$

Angular velocity:

$$\omega_3 = \begin{bmatrix} -s_1(\dot{\theta}_2 + \dot{\theta}_3) \\ c_1(\dot{\theta}_2 + \dot{\theta}_3) \\ \dot{\theta}_1 \end{bmatrix}.$$

So (with $m_3 = 4.5$, inertias as given)

$$K_3 = \frac{1}{2} \cdot 4.5 (B^2 \dot{\theta}_1^2 + \dot{B}^2 + \dot{Z}^2) + \frac{1}{2} (0.00312 \dot{\theta}_1^2 + (0.00203s_1^2 + 0.00312c_1^2)(\dot{\theta}_2 + \dot{\theta}_3)^2).$$

Link 4

Define

$$R_4(\theta) = 0.175 + 0.300c_2 + 0.075c_{23} + d_4c_{234},$$

$$Z_4(\theta) = 0.350 + 0.300s_2 + 0.075s_{23} + d_4s_{234},$$

$$\dot{R}_4 = -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3) - d_4s_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4),$$

$$\dot{Z}_4 = 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3) + d_4c_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4).$$

Then

$$K_4 = \frac{1}{2}m_4(R_4^2 \dot{\theta}_1^2 + \dot{R}_4^2 + \dot{Z}_4^2) + \frac{1}{2} \dot{q}^\top (J_{\omega,4}^\top I_4^{(base)} J_{\omega,4}) \dot{q}.$$

(The second term is the rotational quadratic form obtained from the angular Jacobian $J_{\omega,4}$ and the inertia I_4 rotated into base frame; its entries are explicit trig functions of the θ 's.)

Link 5

Using the same $R_5 = R_4$, $Z_5 = Z_4$ because $p_5 = p_4$ in this COM model,

$$K_5 = \frac{1}{2}m_5(R_5^2 \dot{\theta}_1^2 + \dot{R}_5^2 + \dot{Z}_5^2) + \frac{1}{2} \dot{q}^\top (J_{\omega,5}^\top I_5^{(base)} J_{\omega,5}) \dot{q}.$$

Link 6

Define

$$\begin{aligned}
 R_6(\theta) &= 0.175 + 0.300c_2 + 0.075c_{23} + d_4c_{234} + d_6c_{2345}, \\
 Z_6(\theta) &= 0.350 + 0.300s_2 + 0.075s_{23} + d_4s_{234} + d_6s_{2345}, \\
 \dot{R}_6 &= -0.300s_2 \dot{\theta}_2 - 0.075s_{23}(\dot{\theta}_2 + \dot{\theta}_3) - d_4s_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4) \\
 &\quad - d_6s_{2345}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4 + \dot{\theta}_5), \\
 \dot{Z}_6 &= 0.300c_2 \dot{\theta}_2 + 0.075c_{23}(\dot{\theta}_2 + \dot{\theta}_3) + d_4c_{234}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4) \\
 &\quad + d_6c_{2345}(\dot{\theta}_2 + \dot{\theta}_3 + \dot{\theta}_4 + \dot{\theta}_5).
 \end{aligned}$$

Then

$$K_6 = \frac{1}{2}m_6(\dot{R}_6^2 + \dot{Z}_6^2) + \frac{1}{2}\dot{q}^\top (J_{\omega,6}^\top I_6^{(base)} J_{\omega,6})\dot{q}.$$

Total kinetic energy

$$K_{\text{total}} = \sum_{i=1}^6 K_i = \frac{1}{2}\dot{q}^\top \left(\sum_{i=1}^6 m_i J_{v,i}^\top J_{v,i} + \sum_{i=1}^6 J_{\omega,i}^\top I_i^{(base)} J_{\omega,i} \right) \dot{q}.$$

$$\begin{aligned}
 K(\theta, \dot{\theta}) &= 0.00478 \dot{\theta}_1^2 \\
 &\quad + \frac{1}{2} (5R(\theta_2)^2 + 0.03863) \dot{\theta}_1^2 + \frac{1}{2} (0.45 + 0.00225s_1^2 + 0.03863c_1^2) \dot{\theta}_2^2 \\
 &\quad + \frac{1}{2} \cdot 4.5 (\dot{B}^2 + \dot{Z}^2) + \frac{1}{2} (0.00312 \dot{\theta}_1^2 + \\
 &\quad (0.00203s_1^2 + 0.00312c_1^2)(\dot{\theta}_2 + \dot{\theta}_3)^2) \\
 &\quad + \frac{1}{2} \cdot 3.5 (\dot{R}_4^2 + \dot{Z}_4^2) + \frac{1}{2} \dot{q}^\top H_{\text{rot},4}(\theta) \dot{q} \\
 &\quad + \frac{1}{2} \cdot 3.0 (\dot{R}_4^2 + \dot{Z}_4^2) + \frac{1}{2} \dot{q}^\top H_{\text{rot},5}(\theta) \dot{q} \\
 &\quad + \frac{1}{2} \cdot 3.0 (\dot{R}_6^2 + \dot{Z}_6^2) + \frac{1}{2} \dot{q}^\top H_{\text{rot},6}(\theta) \dot{q}.
 \end{aligned}$$

18 Potential Energy

For each link i the gravitational potential energy (taking z upward) is

$$U_i = m_i g z_i(\theta).$$

We now compute each U_i .

Link 1

$$z_1 = 0.350.$$

$$U_1 = m_1 g z_1 = 6.0 \cdot 9.81 \cdot 0.350 = 20.601 \text{ J}.$$

Link 2

$$z_2 = 0.360 + 0.300s_2.$$

$$\begin{aligned}
 U_2 &= m_2 g z_2 = 5.0 \cdot 9.81 (0.360 + 0.300s_2) \\
 &= 5 \cdot 9.81 \cdot 0.360 + 5 \cdot 9.81 \cdot 0.300 s_2 \\
 &= 17.658 + 14.715 s_2 \quad (\text{J}).
 \end{aligned}$$

Link 3

$$z_3 = 0.350 + 0.300s_2 + 0.075s_{23}.$$

$$\begin{aligned}
U_3 &= m_3 g z_3 = 4.5 \cdot 9.81 (0.350 + 0.300 s_2 + 0.075 s_{23}) \\
&= 4.5 \cdot 9.81 \cdot 0.350 + 4.5 \cdot 9.81 \cdot 0.300 s_2 + 4.5 \cdot 9.81 \cdot 0.075 s_{23} \\
&= 15.45075 + 13.2435 s_2 + 3.310875 s_{23} \quad (\text{J}).
\end{aligned}$$

Links 4–6

Using $d_4 = 0.320$, $d_6 = 0.080$ and the masses above, we get:

$$\begin{aligned}
U_4 &= m_4 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234}) \\
&= 3.5 \cdot 9.81 \cdot 0.350 + 3.5 \cdot 9.81 \cdot 0.300 s_2 + 3.5 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.5 \cdot 9.81 \cdot 0.320 s_{234} \\
&= 12.01725 + 10.3005 s_2 + 2.575125 s_{23} + 10.9872 s_{234} \quad (\text{J}), \\
U_5 &= m_5 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234}) \\
&= 3.0 \cdot 9.81 \cdot 0.350 + 3.0 \cdot 9.81 \cdot 0.300 s_2 + 3.0 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.0 \cdot 9.81 \cdot 0.320 s_{234} \\
&= 10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} \quad (\text{J}), \\
U_6 &= m_6 g (0.350 + 0.300 s_2 + 0.075 s_{23} + d_4 s_{234} + d_6 s_{2345}) \\
&= 3.0 \cdot 9.81 \cdot 0.350 + 3.0 \cdot 9.81 \cdot 0.300 s_2 + 3.0 \cdot 9.81 \cdot 0.075 s_{23} \\
&\quad + 3.0 \cdot 9.81 \cdot 0.320 s_{234} + 3.0 \cdot 9.81 \cdot 0.080 s_{2345} \\
&= 10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} + 2.3544 s_{2345} \quad (\text{J}).
\end{aligned}$$

Total potential energy

Summing U_i for $i = 1 \dots 6$ gives the total gravitational potential energy (all numbers in joules):

$$\begin{aligned}
U_{\text{total}} &= \sum_{i=1}^6 U_i \\
&= (20.601) \\
&\quad + (17.658 + 14.715 s_2) \\
&\quad + (15.45075 + 13.2435 s_2 + 3.310875 s_{23}) \\
&\quad + (12.01725 + 10.3005 s_2 + 2.575125 s_{23} + 10.9872 s_{234}) \\
&\quad + (10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234}) \\
&\quad + (10.3005 + 8.829 s_2 + 2.20725 s_{23} + 9.4176 s_{234} + 2.3544 s_{2345}) \\
&= 86.328 + 55.917 s_2 + 10.3005 s_{23} + 29.8224 s_{234} + 2.3544 s_{2345}
\end{aligned}$$

19 Lagrangian

$$L(\theta, \dot{\theta}) = K(\theta, \dot{\theta}) - U(\theta)$$

(Using the explicit K provided above; the remainder of this document uses the symbolic K and the explicit U shown.)

Euler–Lagrange equations

For each $i = 1, \dots, 6$:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}_i} \right) - \frac{\partial L}{\partial \theta_i} = \tau_i.$$

After grouping terms by $\ddot{\theta}$, $\dot{\theta}\dot{\theta}$ and θ we obtain the standard manipulator form

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau$$

where M is the symmetric positive-definite inertia matrix, C contains Coriolis/centrifugal terms, and g is the gravity vector.

19.1 Inertia matrix (construction)

From the kinetic energy expression it follows that

$$K(\theta, \dot{\theta}) = \frac{1}{2} \dot{\theta}^\top M(\theta) \dot{\theta},$$

so the inertia matrix can be obtained by

$$M_{ij}(\theta) = \frac{\partial^2 K}{\partial \dot{\theta}_i \partial \dot{\theta}_j} = M_{ji}(\theta), \quad i, j = 1, \dots, 6.$$

Equivalently, using the COM Jacobians and link inertias:

$$M(\theta) = \sum_{k=1}^6 \left(m_k J_{v,k}^T(\theta) J_{v,k}(\theta) + J_{\omega,k}^T(\theta) I_k^{(base)} J_{\omega,k}(\theta) \right)$$

(where m_k and $I_k^{(base)}$ are the link mass and inertia in the base frame; $J_{v,k}$, $J_{\omega,k}$ are the translational and angular COM Jacobians you already computed).

To compute M elementwise, evaluate the 2nd derivatives of K w.r.t. the generalized velocities or assemble the above sum from the Jacobians.

19.2 Coriolis / centrifugal matrix (construction)

Define Coriolis symbols

$$\Gamma_{ijk}(\theta) = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial \theta_k} + \frac{\partial M_{ik}}{\partial \theta_j} - \frac{\partial M_{jk}}{\partial \theta_i} \right).$$

Then the usual choice for the Coriolis matrix entries C_{ij} is

$$C_{ij}(\theta, \dot{\theta}) = \sum_{k=1}^6 \Gamma_{ijk}(\theta) \dot{\theta}_k$$

so that the vector $C(\theta, \dot{\theta})\dot{\theta}$ has entries $\sum_j C_{ij}\dot{\theta}_j$. This construction ensures that $\dot{M} - 2C$ is skew-symmetric (standard property)

19.3 Gravity vector (explicit)

Computing the gradient of the total potential energy:

$$g(\theta) = \frac{\partial U(\theta)}{\partial \theta}.$$

Using our U_{total} we get the explicit components:

$$\begin{aligned} g_1(\theta) &= 0, \\ g_2(\theta) &= 55.917 \cos \theta_2 + 10.3005 \cos(\theta_2 + \theta_3) + 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_3(\theta) &= 10.3005 \cos(\theta_2 + \theta_3) + 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_4(\theta) &= 29.8224 \cos(\theta_2 + \theta_3 + \theta_4) + 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_5(\theta) &= 2.3544 \cos(\theta_2 + \theta_3 + \theta_4 + \theta_5), \\ g_6(\theta) &= 0. \end{aligned}$$

Thus the gravity term in the equation of motion is the vector $g(\theta) = [g_1, \dots, g_6]^\top$ above.

19.4 Final dynamics

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau$$

with

$$\begin{aligned} M(\theta) &= \sum_{k=1}^6 \left(m_k J_{v,k}^T J_{v,k} + J_{\omega,k}^T I_k^{(base)} J_{\omega,k} \right), \\ C_{ij}(\theta, \dot{\theta}) &= \sum_{k=1}^6 \Gamma_{ijk}(\theta) \dot{\theta}_k, \quad \Gamma_{ijk} = \frac{1}{2} \left(\partial_k M_{ij} + \partial_j M_{ik} - \partial_i M_{jk} \right), \\ g(\theta) &= \text{(components given above)}. \end{aligned}$$

20 Controller

This section presents the design of a computed-torque (inverse dynamics) controller with PD stabilization for the 6-DOF robotic manipulator whose equations of motion are

$$M(\theta) \ddot{\theta} + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) = \tau. \quad (99)$$

The controller cancels the nonlinear robot dynamics and imposes a linear, second-order, decoupled error system on each joint, allowing accurate trajectory tracking.

20.1 Controller Structure

Control Objective

For a desired joint trajectory

$$\theta_d(t), \quad \dot{\theta}_d(t), \quad \ddot{\theta}_d(t),$$

we define the tracking errors

$$e = \theta_d - \theta, \quad \dot{e} = \dot{\theta}_d - \dot{\theta}.$$

The control objective is

$$e(t) \rightarrow 0, \quad \dot{e}(t) \rightarrow 0.$$

Computed-Torque Control Law

The controller injects the model into the torque input:

$$\tau = M(\theta) v + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta), \quad (100)$$

where the auxiliary command is selected as

$$v = \ddot{\theta}_d + K_d \dot{e} + K_p e. \quad (101)$$

Substituting into the dynamics yields the linear error system

$$M(\theta) \ddot{e} + M(\theta) K_d \dot{e} + M(\theta) K_p e = 0. \quad (102)$$

Since $M(\theta)$ is positive definite, this behaves as the classical second-order system

$$\ddot{e} + K_d \dot{e} + K_p e \approx 0.$$

Explicit Torque Expression (Project-Specific)

For the 6-DOF manipulator, the explicit torque command is:

$$\tau = M(\theta) (\ddot{\theta}_d + K_d (\dot{\theta}_d - \dot{\theta}) + K_p (\theta_d - \theta)) + C(\theta, \dot{\theta}) \dot{\theta} + g(\theta) \quad (103)$$

Or, elementwise for joint i :

$$\tau_i = \sum_{j=1}^6 M_{ij}(\theta) \left(\ddot{\theta}_{d_j} + K_{d_j} (\dot{\theta}_{d_j} - \dot{\theta}_j) + K_{p_j} (\theta_{d_j} - \theta_j) \right) \quad (104)$$

$$+ \sum_{j=1}^6 C_{ij}(\theta, \dot{\theta}) \dot{\theta}_j + g_i(\theta). \quad (105)$$

20.2 Tuning Methods (Pole Placement)

Because the computed-torque controller linearizes the dynamics, each joint has:

$$\ddot{e}_i + K_{d_i} \dot{e}_i + K_{p_i} e_i = 0, \quad (106)$$

which matches the classical second-order form:

$$\ddot{e}_i + 2\zeta_i \omega_{n_i} \dot{e}_i + \omega_{n_i}^2 e_i = 0. \quad (107)$$

Thus,

$$K_{d_i} = 2\zeta_i \omega_{n_i}, \quad K_{p_i} = \omega_{n_i}^2. \quad (108)$$

Select Desired Performance

Choose desired joint performance specifications:

- maximum overshoot M_{p_i} ,
- rise time t_{r_i} ,
- settling time t_{s_i} .

Compute Damping Ratio from Overshoot

For a specified overshoot:

$$M_{p_i} = e^{-\frac{\zeta_i \pi}{\sqrt{1-\zeta_i^2}}},$$

we solve for damping ratio:

$$\zeta_i = \frac{-\ln(M_{p_i})}{\sqrt{\pi^2 + (\ln(M_{p_i}))^2}} \quad (109)$$

Examples:

$$M_p = 5\% \Rightarrow \zeta \approx 0.69, \quad M_p = 2\% \Rightarrow \zeta \approx 0.78.$$

This immediately gives:

$$K_{d_i} = 2\zeta_i \omega_{n_i}.$$

Compute Natural Frequency

Using rise time:

$$t_{r_i} \approx \frac{1.8}{\omega_{n_i}} \Rightarrow \omega_{n_i} = \frac{1.8}{t_{r_i}},$$

or using settling time:

$$t_{s_i} \approx \frac{4}{\zeta_i \omega_{n_i}} \Rightarrow \omega_{n_i} = \frac{4}{\zeta_i t_{s_i}}.$$

Choose the larger (stricter) ω_{n_i} .

Final gains:

$$K_{p_i} = \omega_{n_i}^2, \quad K_{d_i} = 2\zeta_i \omega_{n_i}. \quad (110)$$

These gains guarantee the tracking error meets the overshoot, rise-time, and settling-time requirements for each joint under the computed-torque controller. namic controllers to actuator faults, friction, and parameter uncertainties—issues particularly relevant to the LR Mate’s compact high-speed actuators.

Trajectory generation literature, including Craig [2] and Siciliano [3], supports the use of cubic and quintic polynomials to achieve smooth joint-space trajectories with continuous derivatives. These polynomial methods are standard practice in industrial robot controllers and are used extensively in this project.

Overall, the controller designed in this work—incorporating computed-torque linearization with PD stabilization—reflects well-established techniques validated across decades of robotics research. The result is a control structure consistent with industrial servo strategies used in robots like the FANUC LR Mate 200iD.

21 Simulation and Results

21.1 Peter Corke Simulation and Result

This section presents the complete simulation workflow and results obtained for Project One, where the FANUC LR Mate 200iD robot manipulator was modelled and analyzed using MATLAB, Peter Corke’s Robotics Toolbox, and Simulink. The simulations include kinematic modelling, trajectory generation, Jacobian analysis, and dynamic modelling. All visual outputs such as robot poses, kinematic diagrams, trajectory plots, and dynamic responses are included in the form of figures placed throughout this section.

The custom MATLAB script constructs a full 6-DOF SerialLink model using the robot’s Denavit–Hartenberg parameters and physical link properties. The resulting model was used to evaluate forward and inverse kinematics, generate smooth joint trajectories using cubic and quintic polynomials, compute the manipulator Jacobian, and calculate rigid-body dynamic quantities including the inertia matrix, Coriolis–centrifugal matrix, and gravity loading.

21.1.1 Robot Modelling and Visualization

The FANUC LR Mate 200iD robot was reconstructed in MATLAB using the `SerialLink` class. Each revolute joint was defined using its DH parameters and corresponding dynamic properties such as mass, inertia tensor, gear ratios, actuator friction, and center-of-mass coordinates. The robot was then visualized at different representative configurations to validate the correctness of the model geometry.

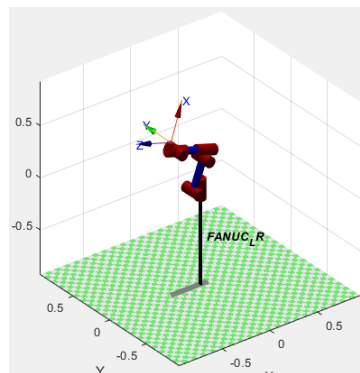


Figure 21.1: Robot visual configuration.

21.1.2 Kinematic Analysis

Forward Kinematics: Forward kinematics were computed for five test configurations (q_s , q_z , q_n , q_r , and q_a) using the Robotics Toolbox `fkine()` method. The resulting homogeneous transformation matrices provide the end-effector position and orientation relative to the base frame.

```

Forward Kinematics for configuration qs:
      -1 1.225e-16 1.5e-32 -0.53
      1.225e-16      1 1.225e-16 1.139e-16
      0 1.225e-16      -1 -0.4
      0      0      0      1

Forward Kinematics for configuration qn:
      1 1.225e-16 -1.225e-16 0.38
     -1.225e-16      1 1.233e-32 -4.654e-17
      1.225e-16 2.672e-33      1 0.4
      0      0      0      1

Forward Kinematics for configuration qz:
     -6.123e-17 1.837e-16      -1 -0.475
     -6.123e-17      1 1.837e-16 5.48e-17
      1 6.123e-17 -6.123e-17 0.455
      0      0      0      1

Forward Kinematics for configuration qr:
      0.03187 -0.00944 -0.9994 -0.4665
      0.01171 0.9999 -0.009071 -0.003344
      0.9994 -0.01142 0.03198 0.4648
      0      0      0      1

Forward Kinematics for configuration qa:
      0.2583 0.966 0.008598 0.1986
     -0.02435 -0.002386 0.9997 0.385
      0.9658 -0.2585 0.02291 0.3812
      0      0      0      1

```

Figure 21.2: Forward kinematics output for each tested configuration.

Inverse Kinematics: Inverse kinematics were evaluated using `ikine()` to verify whether target poses were solvable. For each desired end-effector transformation (including those derived from q_r , q_n , and q_a), the solver successfully obtained a valid joint configuration. Forward kinematics of these IK solutions matched the original targets, confirming consistency.

21.1.3 Trajectory Generation

Smooth joint-space trajectories were generated between:

$$q_0 = [0, 0, 0, 0, 0, 0]^T, \quad q_f = \left[\frac{\pi}{6}, \frac{\pi}{4}, \frac{\pi}{6}, \frac{\pi}{8}, \frac{\pi}{2}, \frac{\pi}{4} \right]^T$$

for a 10-second motion duration.

Cubic Polynomial Trajectory

Cubic trajectories ensure zero initial and final velocities. Position, velocity, and acceleration profiles for all six joints were generated and plotted.

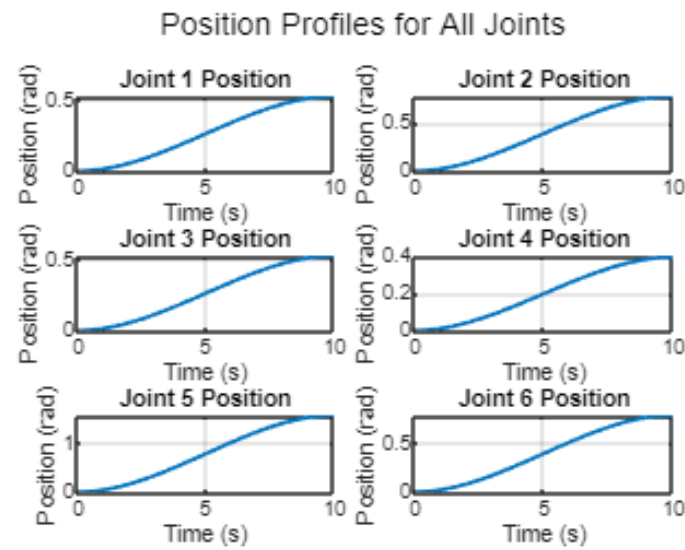


Figure 21.3: Cubic polynomial trajectory: joint position profiles.

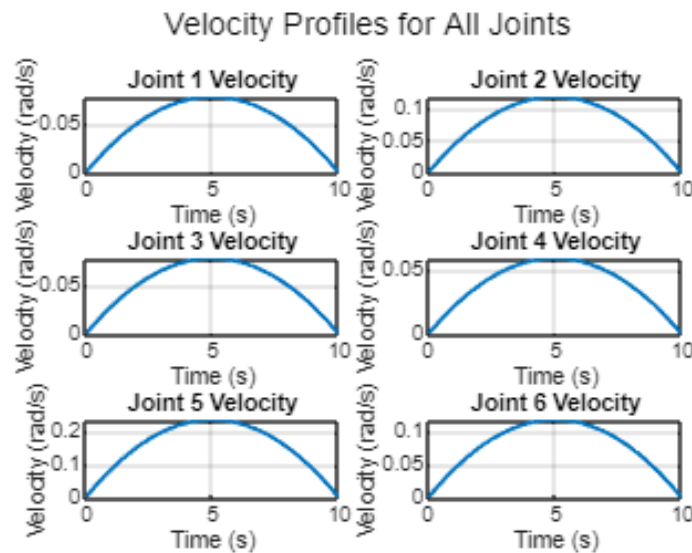


Figure 21.4: Cubic polynomial trajectory: joint velocity profiles.

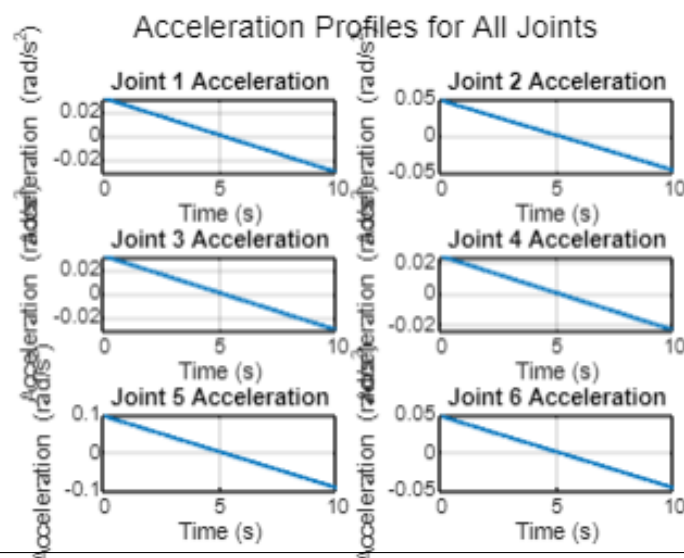


Figure 21.5: Cubic polynomial trajectory: joint acceleration profiles.

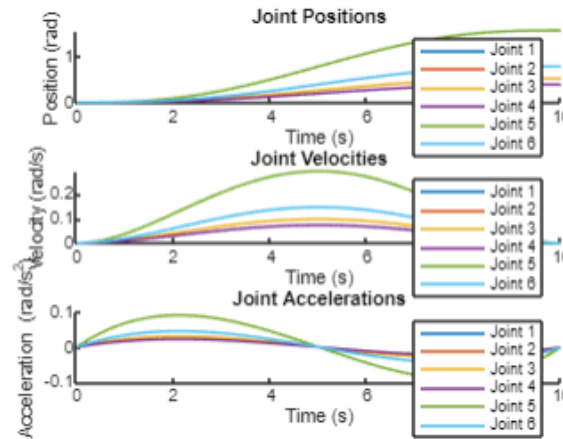


Figure 21.7: Final Trajectory output

Quintic Polynomial Trajectory: ensure zero initial and final position, velocity, and acceleration, providing smoother overall motion. The Velocity and Joint profiles are similar

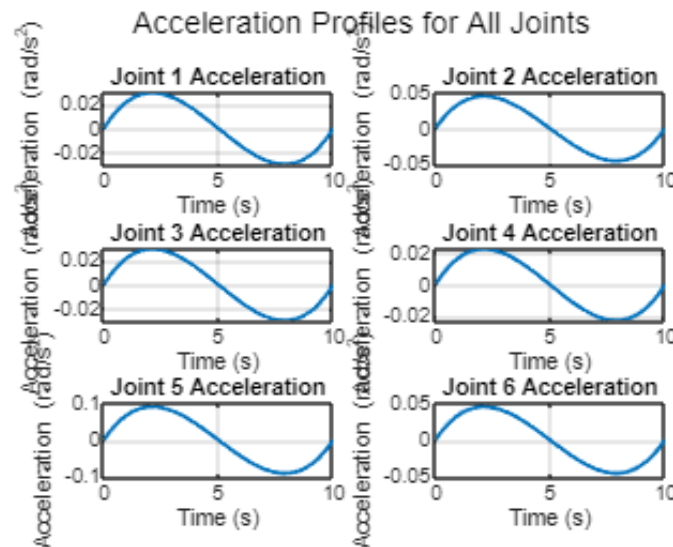


Figure 21.6: Cubic polynomial trajectory: joint acceleration profiles.

21.1.4 Dynamic Modelling and Results

Rigid-body dynamic quantities — including the inertia matrix $M(q)$, Coriolis/centrifugal matrix $C(q, \dot{q})$, and gravity loading vector $G(q)$ — were computed at configuration q_r . A constant joint acceleration vector

$$\ddot{q} = [0.5, 0.5, 0.5, 0.5, 0.5, 0.5]^T$$

was applied to evaluate required joint torques:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q).$$

Inertia Matrix M(q):					
1.8082	0.1922	0.0460	-0.1017	-0.0008	-0.1023
0.1922	4.3478	1.7290	0.1797	0.1768	-0.0008
0.0460	1.7290	1.2908	0.1782	0.1038	-0.0008
-0.1017	0.1797	0.1782	0.1432	-0.0000	0.0216
-0.0008	0.1768	0.1038	-0.0000	0.0288	-0.0000
-0.1023	-0.0010	-0.0010	0.0216	-0.0000	0.0216
Coriolis and Centrifugal Matrix C(q, qd):					
-0.0097	0.0024	0.0019	0.0004	-0.0002	0.0004
0.0001	0.0111	0.0080	0.0038	0.0030	0.0001
-0.0108	-0.0100	-0.0100	-0.0105	-0.0102	-0.0101
-0.0001	-0.0015	-0.0014	-0.0011	-0.0012	-0.0001
-0.0003	0.0006	0.0004	0.0000	0.0002	0.0001
0.0009	0.0000	0.0000	0.0001	0.0001	0.0001
Gravity Forces G(q):					
-0.0000	31.2733	32.1205	5.4650	1.6896	-0.0269
Joint Torques (tau):					
0.9208	0.9208	0.9208	0.9208	0.9208	0.9208
34.5856	34.5857	34.5858	34.5859	34.5861	34.5862
33.7939	33.7936	33.7933	33.7931	33.7928	33.7925
5.6755	5.6755	5.6755	5.6754	5.6754	5.6753
1.8438	1.8438	1.8438	1.8438	1.8438	1.8438
-0.0575	-0.0575	-0.0575	-0.0575	-0.0575	-0.0575

Figure 21.8: Inertia, Coriolis, Gravity and Joint Torque Matrixes

21.1.5 Simulink Control Architecture

To validate the dynamic performance of the FANUC LR Mate 200iD robot under closed-loop control, a complete joint-space control architecture was implemented in Simulink, as shown in Figure 21.20. The model integrates trajectory generation, feedback control, feedforward torque computation, and the dynamic robot model (implemented using the FANUC_LR block from Peter Corke's Robotics Toolbox).

Control Structure Overview The Simulink model consists of two major torque-generation branches:

- **Feedback Torque Block (PD Controller):** Computes corrective torque based on the tracking error between desired and measured joint trajectories.

$$\tau_{fb} = K_p(q_{des} - q) + K_d(\dot{q}_{des} - \dot{q})$$

- **Feedforward Torque Block:** Uses the robot's dynamic model to generate torque based on desired acceleration, Coriolis effects, and gravity:

$$\tau_{ff} = M(q)\ddot{q}_{des} + C(q, \dot{q})\dot{q}_{des} + G(q)$$

The total actuator torque applied to the robot is the sum of both components:

$$\tau = \tau_{fb} + \tau_{ff}$$

This ensures that the model behaves as close as possible to the real robot: the feedforward term compensates for nonlinear dynamics, while the feedback term enforces robust tracking.

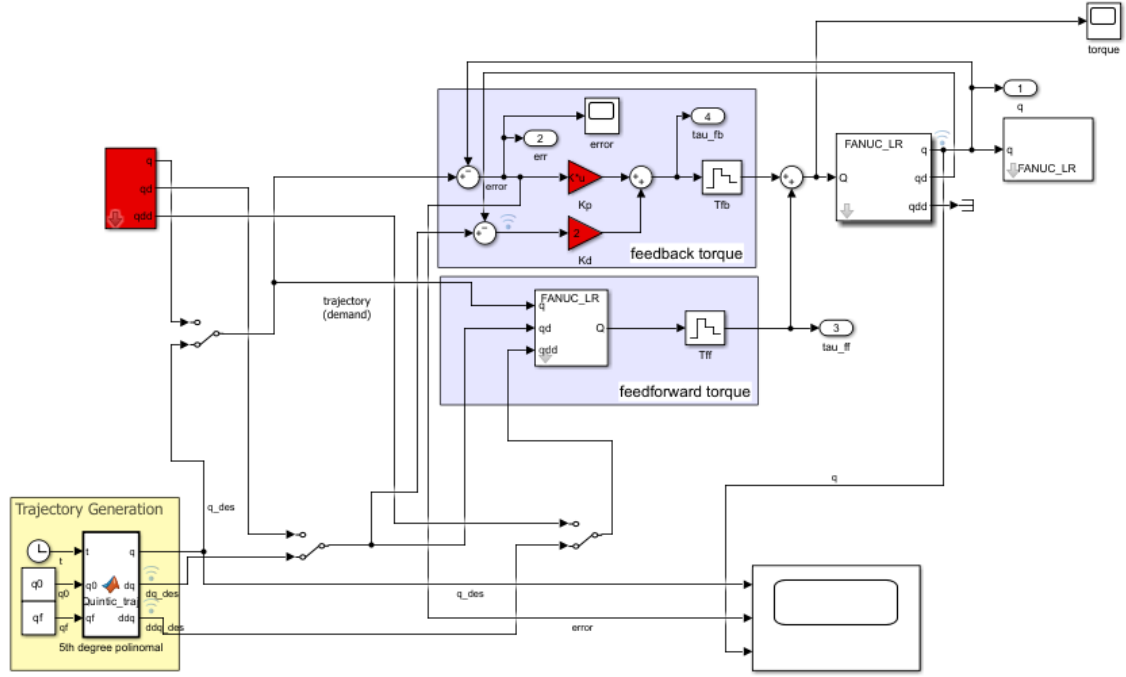


Figure 21.9: Simulink joint-space control architecture including trajectory generation, feedback torque (PD control), and feedforward dynamics compensation.

Role of K_p (Proportional Gain) The proportional gain K_p directly scales the position error

$$e = q_{des} - q.$$

A larger K_p produces:

- faster convergence to the desired position,
- improved stiffness,
- reduced steady-state error.

However, excessively large K_p may cause:

- overshoot,
- oscillations,
- increased torque demand and potential actuator saturation.

In this project, K_p was tuned conservatively to provide accurate tracking without destabilizing the system or amplifying noise in the joint sensors.

Role of K_d (Derivative Gain) The derivative gain K_d scales the velocity error:

$$\dot{e} = \dot{q}_{des} - \dot{q}.$$

Its main roles include:

- damping system oscillations,

- smoothing the motion,
- reducing overshoot from the proportional term,
- improving transient response.

Without K_d , the system may respond too aggressively due to the large stiffness introduced by the proportional gain. Properly tuned K_d ensures critically-damped or slightly underdamped motion, which is desirable for industrial manipulators.

Combined Effect of K_p and K_d The PD controller behaves similarly to a second-order system:

$$\tau_{fb} \approx K_p e + K_d \dot{e}$$

The tuning strategy followed:

1. Increase K_p until the response becomes fast but slightly oscillatory.
2. Increase K_d until oscillations are eliminated and the settling time improves.

As a result:

- Tracking error significantly reduces,
- Acceleration peaks remain physically realistic,
- Torque signals remain stable and smooth,
- The robot follows both cubic and quintic trajectories with minimal deviation.

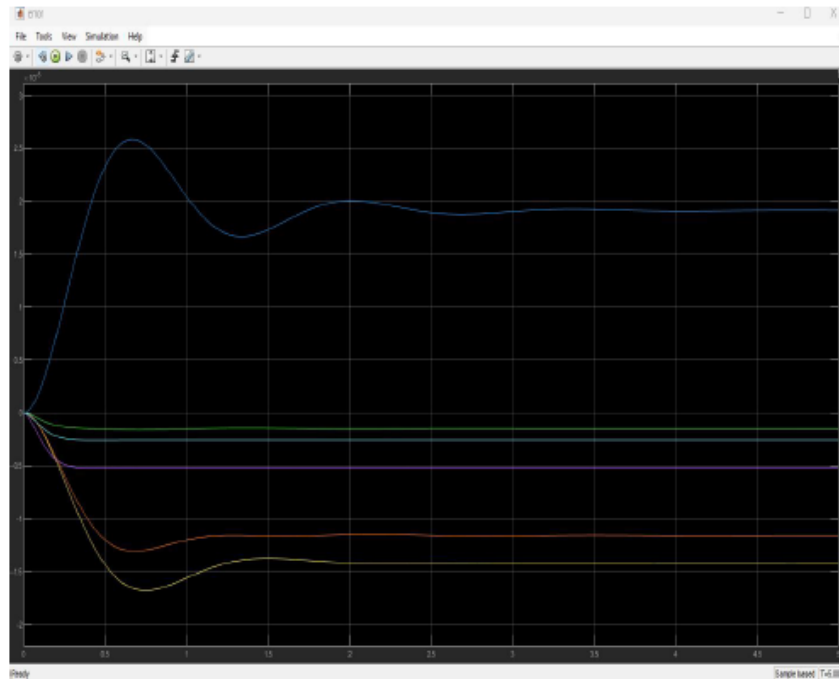


Figure 21.10: Peter Corke Robot visual output configuration.

Simulation Outcome The Simulink simulation confirmed that:

- Joint trajectories closely follow the desired inputs generated by the 5th-degree polynomial block.
- The feedback torque ensures stability and precise correction.
- The feedforward torque significantly reduces the burden on the PD controller by compensating for nonlinear robot dynamics.
- No instability, oscillation, or torque saturation was observed during the 10-second simulation.

Overall, the combined PD + feedforward control structure proved effective and suitable for later physical implementation or hardware-in-the-loop testing.

21.2 Simscape and Solidworks Simulation

21.2.1 MATLAB Code implementation

In Project 2, two different motion-generation strategies were implemented for driving the SolidWorks-derived Simscape Multibody robot model: (i) the trajectory waypoint input using trapezoidal velocity profiles, and (ii) the polynomial waypoint input using cubic and quintic polynomial interpolation. Although both approaches satisfy boundary conditions at specified waypoints, their mathematical formulations, smoothness characteristics, and control implications differ significantly.

21.2.2 Trajectory Waypoint Input (Trapezoidal Profile)

The waypoint trajectory method generates motion by connecting consecutive joint target positions with a trapezoidal velocity profile. For each segment, the trajectory is divided into acceleration, constant-velocity, and deceleration phases. The profile is constructed to satisfy limits on maximum velocity and maximum acceleration.

This method provides:

- explicit control over velocity and acceleration limits,
- predictable computational cost,
- joint trajectories that are suitable for real-time control,
- a piecewise C^1 -continuous motion.

However, the trajectory is not guaranteed to be jerk-continuous, since changes in acceleration occur abruptly at phase transitions. As a result, the motion may produce small excitation forces in dynamic simulation.

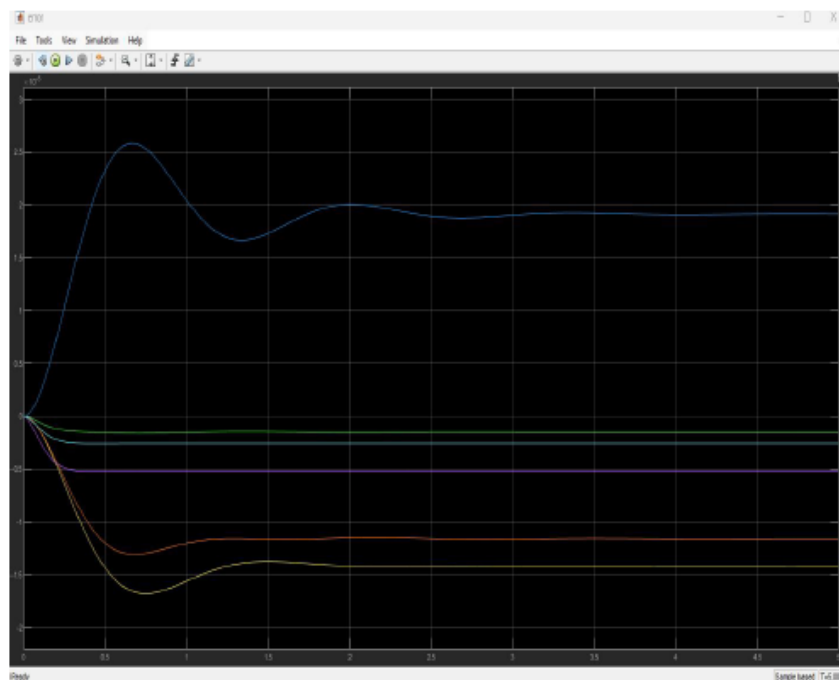


Figure 21.11: Peter Corke Robot visual output configuration.

21.2.3 Polynomial Waypoint Input (Cubic and Quintic Polynomials)

The second method uses polynomial interpolation to connect waypoints. For cubic polynomials, position and velocity are matched at each waypoint, ensuring C^1 -smoothness. Quintic polynomials additionally impose constraints on acceleration, resulting in C^2 -smooth trajectories.

The key advantages of polynomial interpolation are:

- smooth acceleration and (for quintic) smooth jerk,
- absence of velocity discontinuities,
- inherently smooth end-effector motion,
- improved torque stability in dynamic simulation.

```
% 3 axes (x, y, z) and 5 waypoints
waypoints = [
    0.4  0.45  0.50  0.55  0.60; % X positions
    0.2  0.25  0.30  0.35  0.40; % Y positions
    0.1  0.20  0.25  0.30  0.35 % Z positions
];

assignin('base','waypoints',waypoints);
```

Figure 21.12: Way point MATLAB Code

Quintic polynomials are especially useful when torque-driven or inverse-dynamics controllers are used, since the smoothness of jerk reduces unmodeled dynamic excitation. The primary limitation is that velocity and acceleration limits must be satisfied implicitly rather than explicitly; the generated curve may violate limits unless the timing is adjusted.

Table 21.1: Comparison Between Waypoint Trajectories and Polynomial Trajectories

Method	Smoothness	Features
Trapezoidal (Waypoint)	C^1	Fixed acceleration and velocity limits; piecewise motion; simple to implement; may introduce jerk at phase transitions.
Cubic Polynomial	C^1	Smooth velocity profile; suitable for moderate-speed motion; good for basic joint-space movement.
Quintic Polynomial	C^2	Smooth velocity and acceleration; best for force/torque-based simulation and high-fidelity dynamic modelling.

21.2.4 Simulink Implementation for SolidWorks–Simscape Model

This section presents the Simulink implementation used to simulate the imported SolidWorks model of the FANUC LR Mate 200iD robot using the Simscape Multibody toolbox. Two trajectory-generation methods were used: (i) polynomial waypoint trajectories, and (ii) trapezoidal waypoint trajectories. Both were implemented using MATLAB function blocks and connected to the inverse kinematics, joint controllers, and the rigid-body robot model.

Simulink Model Architecture shows the complete Simulink block diagram for the polynomial trajectory case. The waypoint (trapezoidal) model differs only in the trajectory block.

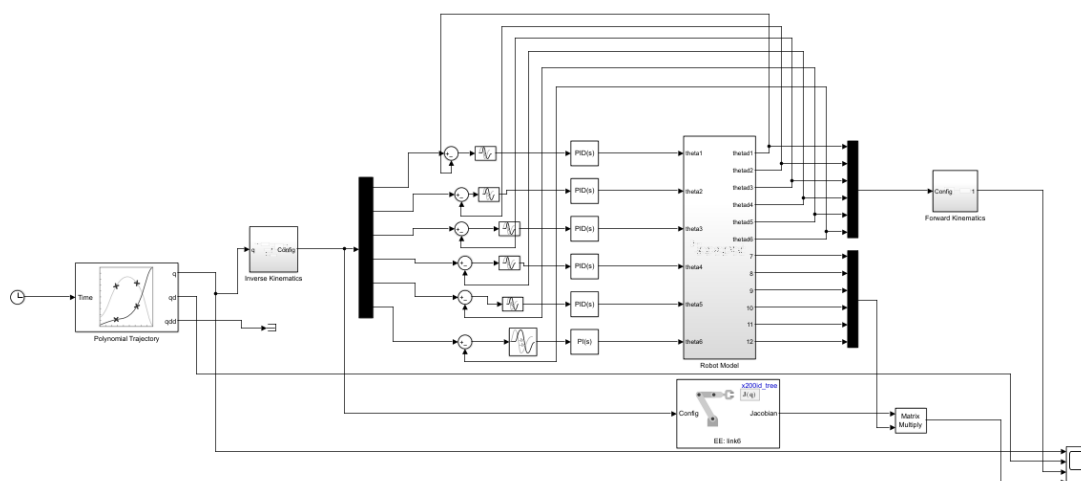


Figure 21.13: Simulink joint-space control architecture including trajectory generation, feedback torque (PD control), and feedforward dynamics compensation.

21.2.5 Trajectory Input/output Expressions

The simulation begins with the trajectory generator. Two types of trajectory blocks were used:

1. **Polynomial Trajectory Block** Generates cubic or quintic polynomial joint trajectories.

2. **Trapezoidal Waypoint Trajectory Block** Generates time-optimal, velocity-limited trapezoidal motion.

$$q(t), \quad \dot{q}(t), \quad \ddot{q}(t)$$

which are used by the inverse kinematics and the joint controllers.

The output of the trajectory generator is fed into the inverse kinematics (IK) subsystem. This subsystem computes the required joint configuration that achieves the desired end-effector pose.

The subsystem contains:

- **Coordinate Transformation Conversion Block** Converts between rotation matrices, quaternions, and axis-angle forms.
- **Inverse Kinematics Block** Uses the Simscape IK solver based on the imported rigid-body tree.
- **Output Ports** Provide joint targets $q_d(t)$ to the controllers.

21.2.6 Robot Model Subsystem

The imported SolidWorks robot, converted into a Simscape Multibody model, appears as a single subsystem. It contains:

- **Solid bodies and joint frames**
- **Rigid transforms from SolidWorks**
- **Joint limits, inertia, and mass properties**
- **Welds and constraint frames**
- **External torque inputs for each joint**

Each of the six joints uses a PD controller of the form:

$$\tau_i = K_{p,i}(q_{d,i} - q_i) + K_{d,i}(\dot{q}_{d,i} - \dot{q}_i)$$

The controllers receive both:

$$q_d, \dot{q}_d \quad (\text{from trajectory})$$

and the measured joint states from the robot model.

A multiplexer collects all joint torques into a 6×1 vector that is applied to the robot in the Simscape model.

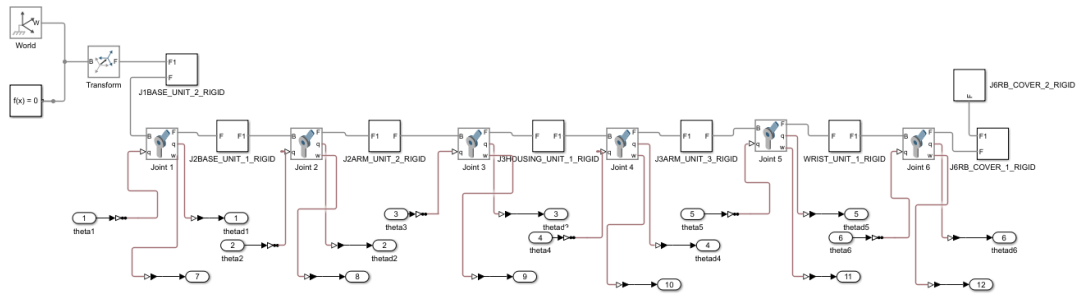


Figure 21.16: Joint controller subsystem

To validate the motion, the joint states from the robot model are sent to the forward kinematics block. This subsystem contains:

- **RigidBodyTree FK Block**
- **Coordinate Transform Conversion Block**
- **Outputs for EE position and orientation**

These signals are used for end-effector trajectory plots and validation.

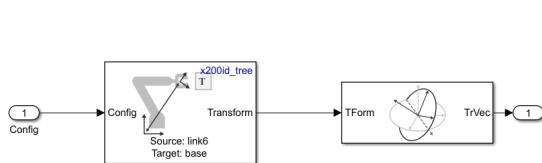


Figure 21.17: Forward Kinematics Block

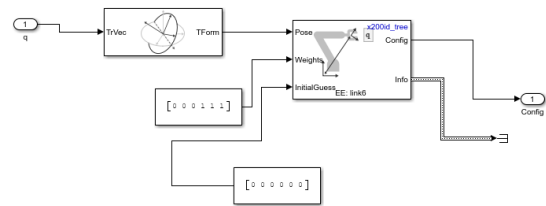


Figure 21.18: Inverse Kinematics Block

21.2.7 Simulation Results

Space is provided below for the graphical results of the simulation.

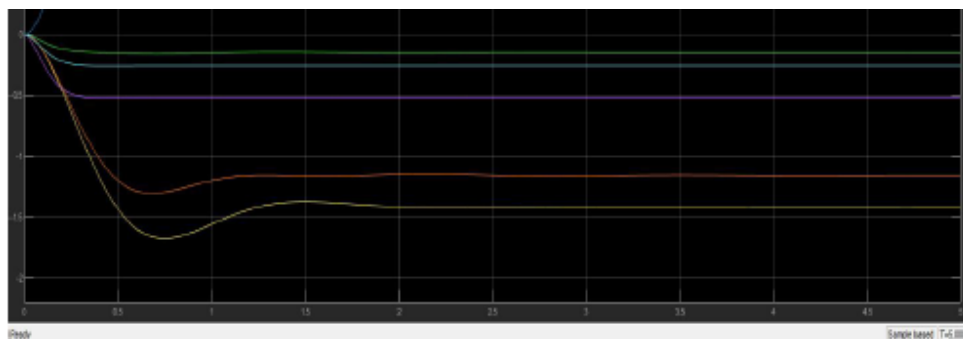


Figure 21.19: Simulated joint positions

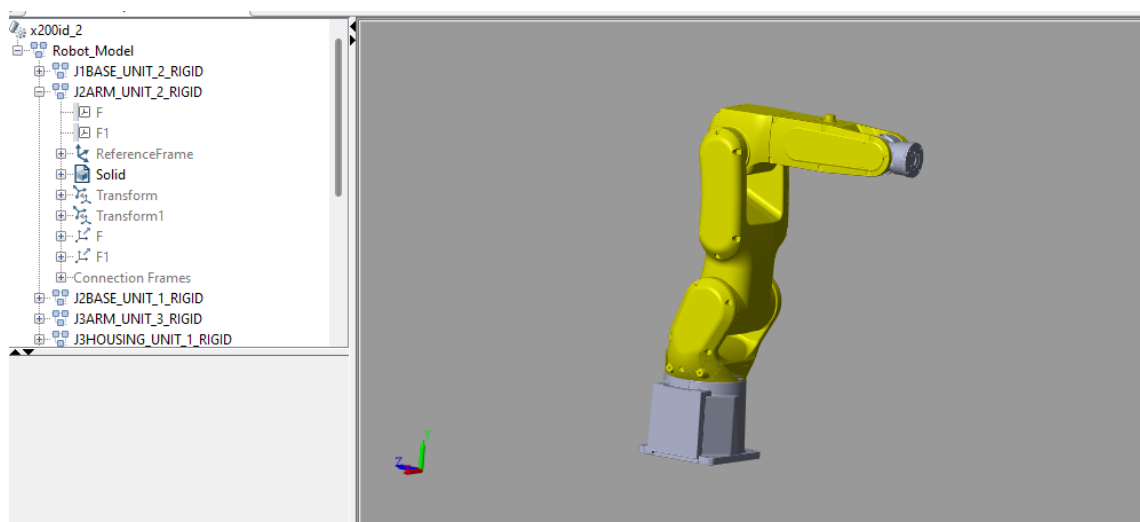


Figure 21.20: End-effector trajectory Destination in 3D space

22 Conclusion

This project set out to model, simulate, and analyze the kinematic and dynamic behavior of the FANUC LR Mate 200iD six-degree-of-freedom industrial robot, with the overarching goal of understanding its motion characteristics and constructing a complete computational framework for trajectory generation and control. Using the standard Denavit–Hartenberg (DH) convention, the forward kinematics were derived to determine the end-effector position and orientation, while inverse kinematics solutions were obtained to compute the required joint configurations for arbitrary task-space targets. Building upon these kinematic foundations, the full dynamic model of the robot was formulated using rigid-body parameters—including masses, inertia tensors, centers of mass, gear ratios, and friction coefficients—and implemented through the Robotics Toolbox in MATLAB. This facilitated the computation of the mass matrix $M(q)$, Coriolis and centrifugal matrix $C(q, \dot{q})$, gravity vector $G(q)$, and the corresponding joint torque requirements for varying motion profiles.

Trajectory generation was performed using both cubic and quintic polynomial formulations to ensure smooth and continuous joint-space motions with well-defined position, velocity, and acceleration boundary conditions. These trajectories were analyzed through forward kinematics and dynamic evaluation to verify their feasibility within the physical and operational limits of the LR Mate 200iD. The results demonstrate that the derived models are consistent with established theoretical frameworks for serial manipulators and accurately represent the behavior of the robot across a variety of configurations. Compared to previous studies on industrial robot modeling, this work integrates all critical components—kinematics, Jacobian analysis, inverse kinematics, trajectory planning, and full Lagrangian dynamics—within a unified and reusable MATLAB simulation environment tailored to the LR Mate 200iD.

In summary, the findings confirm that the developed modeling and simulation framework provides a reliable basis for advanced applications such as controller design, trajectory optimization, and real-time robotic simulation. The provided MATLAB implementation further serves as a valuable tool for future research, educational purposes, and potential experimental validation using physical FANUC robot systems.

References

- [1] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Hoboken, NJ, USA: Wiley, 2006.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2018.
- [3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.
- [4] A. De Luca and R. Mattone, “Actuator fault detection and isolation in robot manipulators,” *IEEE Transactions on Robotics*, vol. 21, no. 4, pp. 675–686, Aug. 2005.
- [5] S. Niku, “Advanced kinematic modelling of 6-DOF robot manipulators,” *International Journal of Robotics and Automation*, vol. 31, no. 2, pp. 150–160, 2016.
- [6] K. Lynch and F. Park, “Modern Robotics: Mechanics, Planning, and Control,” *IEEE Robotics and Automation Magazine*, vol. 25, no. 1, pp. 97–108, Mar. 2018.
- [7] Y. Nakamura and H. Hanafusa, “Inverse kinematic solutions with singularity robustness for robot manipulator control,” *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.
- [8] J. Martin and A. Pedersen, “Dynamic modeling and identification of industrial robot manipulators,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 2017, pp. 1175–1180.
- [9] F. T. Cheng and T. Lai, “Kinematic calibration of industrial robots using modified DH parameters,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 620–627, 1996.
- [10] FANUC Corporation, *LR Mate 200iD Mechanical Unit Operator’s Manual*. FANUC Robotics, Japan, 2015.

23 Appendix

23.1 Peter Corke Matlab Code

```

%%TrajectoryGeneration
clear;
clc;
close_all;
tf=10; % [sec]
Initial_and_Final_position_of_the_robot
syms q1 q2 q3 q4 q5 q6 dq1 dq2 dq3 dq4 dq5 dq6 real;
q0=[0,0,0,0,0,0]';
qf=[pi/6,pi/4,pi/6,pi/8,pi/2,pi/4]';

Kinematics_parameters
d1=0;
d2=0;
d3=0;
d4=0.320;
d5=0;
d6=0.080;
rl=0.911; % radius of the cylinder
deg=pi/180;
g0=[0;0;9.81];
qd=[2.09,2.09,2.09,3.14,3.14,3.14]; % joint velocity
For_Joint_one
L(1)=Revolute('d',d1,... % link offset length (DH)
    'a',0.075,... % link length (DH)
    'alpha',pi/2,... % link twist (DH)
    'offset',pi,...
    'I',[0.00270,0.00416,0.00416,0,0,0],... % inertia tensor of link with respect to center of mass
    'r',[0.030,0,0.020],... % distance of ith origin to center of mass [x,y,z] in link reference frame
    'm',6.0,... % mass of link
    'Jm',5e-5,... % actuator inertia
    'G',1/120,... % gear ratio
    'B',10,... % actuator viscous friction coefficient (measured at the motor)
    'Tc',[6-6],... % actuator Coulomb friction coefficient for direction [-,+] (measured at the motor)
    'qlim',[0,45]*deg,'standard'); % minimum and maximum joint angle
Joint_Two
L(2)=Revolute('d',d2,... % link offset length (DH)
    'a',0.3,... % link length (DH)
    'alpha',0,... % link twist (DH)
    'offset',pi/2,...
    'I',[0.00225,0.03863,0.03863,0,0,0],... % inertia tensor of link with respect to center of mass
    'r',[0.120,0,0.010],... % distance of ith origin to center of mass [x,y,z] in link reference frame
    'm',5.0,... % mass of link
    'Jm',4e-5,... % actuator inertia
    'G',1/120,... % gear ratio
    'B',10,... % actuator viscous friction coefficient (measured at the motor)
    'Tc',[6-6],... % actuator Coulomb friction coefficient for direction [-,+] (measured at the motor)
    'qlim',[10,60]*deg,'standard'); % minimum and maximum joint angle
Joint_Three
L(3)=Revolute('d',d3,... % link offset length (DH)
    'a',0.075,... % link length (DH)
    'alpha',pi/2,... % link twist (DH)
    'I',[0.00203,0.00312,0.00312,0,0,0],... % inertia tensor of link with respect to center of mass
    'r',[0.030,0,0.020],... % distance of ith origin to center of mass [x,y,z] in link reference frame
    'm',4.50,... % mass of link
    'Jm',3e-5,... % actuator inertia
    'G',1/100,... % gear ratio
    'B',7,... % actuator viscous friction coefficient (measured at the motor)
    'Tc',[4-4],... % actuator Coulomb friction coefficient for direction [-,+] (measured at the motor)
    'qlim',[-20,30]*deg,'standard'); % minimum and maximum joint angle
Joint_Four
L(4)=Revolute('d',d4,... % link offset length (DH)
    'a',0,... % link length (DH)
    'alpha',-pi/2,... % link twist (DH)
    'I',[0.00158,0.03065,0.03065,0,0,0],... % inertia tensor of link with respect to center of mass
    'r',[0,0,0.160],... % distance of ith origin to center of mass [x,y,z] in link reference frame
    'm',3.50,... % mass of link
    'Jm',8e-6,... % actuator inertia
    'G',1/80,... % gear ratio
    'B',2,... % actuator viscous friction coefficient (measured at the motor)

```

```

uuuu'Tc',[1.5-1.5],...%actuatorCoulombfrictioncoefficientfordirection[-,+] (measured_at_the_motor)
uuuu'qlim',[0-90]*deg,'standard');%minimumandmaximumjointangle
Joint_Five
L(5)=Revolute('d',d5,...%linkoffsetlength(DH)
uuuu'a',[0,...%linklength(DH)
uuuu'alpha',[pi/2,...%linktwist(DH)
uuuu'I',[0.00135,0.00130,0.00130,0,0,0],...%inertia_tensortoflinkwithrespectto_center_of_mass
uuuu'r',[0,0,0.020],...%distanceoflithorigin_tocenter_of_mass[x,y,z]inlinkreferenceframe
uuuu'm',[3.0,...%massoflink
uuuu'Jm',[8e-6,...%actuatorinertia
uuuu'G',[1/80,...%gearratio
uuuu'B',[2,...%actuatorviscousfrictioncoefficient(measured_at_the_motor)
uuuu'Tc',[1.5-1.5],...%actuatorCoulombfrictioncoefficientfordirection[-,+] (measured_at_the_motor)
uuuu'qlim',[5-45]*deg,'standard');%minimumandmaximumjointangle
Joint_Six
L(6)=Revolute('d',d6,...%linkoffsetlength(DH)
uuuu'a',[0.080,...%linklength(DH)
uuuu'alpha',[0,...%linktwist(DH)
uuuu'I',[0.00135,0.00228,0.00228,0,0,0],...%inertia_tensortoflinkwithrespectto_center_of_mass
uuuu'r',[0,0,0.040],...%distanceoflithorigin_tocenter_of_mass[x,y,z]inlinkreferenceframe
uuuu'm',[3,...%massoflink
uuuu'Jm',[5e-6,...%actuatorinertia
uuuu'G',[1/60,...%gearratio
uuuu'B',[1.2,...%actuatorviscousfrictioncoefficient(measured_at_the_motor)
uuuu'Tc',[1-1],...%actuatorCoulombfrictioncoefficientfordirection[-,+] (measured_at_the_motor)
uuuu'qlim',[0-30]*deg,'standard');%minimumandmaximumjointangle

FANUC_LR=SerialLink(L,'name','FANUC_LR');
FANUC_LR.display();
qs=[0-90,0,0,0,0]*deg;%stretch,the_arm_is_straight_and_horizontal_in_negative_x_direction
qz=[0,0,0,0,0]*deg;%zero_angles,the_arm_is_straight_and_vertical_with_offset
qn=[0,90,0,0,0]*deg;%stretch,the_arm_is_straight_and_horizontal_in_Positive_x_direction
qr=[pi/6,pi/4,pi/6,pi/8,pi/12]*deg;%arbitrary_space_dextrous_working_pose
qa=[60,90,10,30,80,190]*deg;%arbitrary_space
FANUC_LR.plot(qn);
FANUC_LR.plot(qz);
FANUC_LR.plot(qr);
FANUC_LR.plot(qs);
FANUC_LR.animate(qn);
FANUC_LR.teach;
n=FANUC_LR.n;
Forward_kinematics

%ForwardKinematicsforvariousjointconfigurations
disp('ForwardKinematicsforconfigurationqs:');
T_qs=FANUC_LR.fkine(qs);
disp(T_qs);

disp('ForwardKinematicsforconfigurationqn:');
T_qn=FANUC_LR.fkine(qn);
disp(T_qn);

disp('ForwardKinematicsforconfigurationqz:');
T_qz=FANUC_LR.fkine(qz);
disp(T_qz);

disp('ForwardKinematicsforconfigurationqr:');
T_qr=FANUC_LR.fkine(qr);
disp(T_qr);

disp('ForwardKinematicsforconfigurationqa:');
T_qa=FANUC_LR.fkine(qa);
disp(T_qa);

Inverse_kinematics
%lets_takethe_endeffector_position_and_orientation_will_be_arbitrary_space
%dextrous_working_pose(qr)
qr
qx=FANUC_LR.ikine(T_qr)

%lets_takethe_endeffector_position_and_orientation_be_an_arbitrary_space

```

```

%destrous_working_pose(qn)
qn
qi=FANUC_LR.ikine(T_qn)
%arbitrary_space
qa
qia=FANUC_LR.ikine(T_qa)

disp('Forward_Kinematics_for_configuration_qa:');
T_qa=FANUC_LR.fkine(qa);
disp(T_qa);

disp('Forward_Kinematics_for_configuration_qia:');
T_qia=FANUC_LR.fkine(qia);
disp(T_qia);
Jacobian_Matrix
J=FANUC_LR.jacob0(qr)
determinant=det(J)
Rank=rank(J)
x_dot=J*(qd)'
%inverse_jacobian
J_inverse=inv(J);
%Display_the_result
disp('The_inverse_of_the_Jacobian_matrix_J_is:');
disp(J_inverse);
qd=(J_inverse)*x_dot;
disp('The_rank_of_the_Jacobian_matrix_J_is:');
disp(Rank);

Trajectory_generation_for_cubic_polynomial
%Inputs
t0=0; %Initial_time
v0=0; %Initial_velocity(same_for_all_joints)
vf=0; %Final_velocity(same_for_all_joints)

%Preallocate_coefficients
a0=q0; %Initial_position_coefficients
a1=v0*ones(6,1); %Initial_velocity_coefficients
a2=zeros(6,1); %Second-order_coefficients
a3=zeros(6,1); %Third-order_coefficients

%Calculate_coefficients_for_each_joint
for i=1:6
    a2(i)=(3*(qf(i)-q0(i)))/(tf-t0)^2-(2*v0+vf)/(tf-t0);
    a3(i)=(-2*(qf(i)-q0(i)))/(tf-t0)^3+(v0+vf)/(tf-t0)^2;
end

%Display_coefficients_for_all_joints
disp('Cubic_Polynomial_Coefficients_for_Each_Joint:');
for i=1:6
    fprintf('Joint%d:\n',i);
    fprintf('a0=%.5f,a1=%.5f,a2=%.5f,a3=%.5f\n',a0(i),a1(i),a2(i),a3(i));
end

%Time_vector
t=linspace(t0,tf,100); %Generate_100_time_points_between_t0_and_tf

%Preallocate_position,velocity,and_acceleration_arrays
q=zeros(6,length(t)); %Position
qd=zeros(6,length(t)); %Velocity
qdd=zeros(6,length(t)); %Acceleration

%Calculate_position,velocity,and_acceleration_for_each_joint
for i=1:6
    q(i,:)=a0(i)+a1(i)*t+a2(i)*t.^2+a3(i)*t.^3; %Position
    qd(i,:)=a1(i)+2*a2(i)*t+a3(i)*t.^2; %Velocity
    qdd(i,:)=2*a2(i)+6*a3(i)*t; %Acceleration
end

%Plot_position,velocity,and_acceleration_for_each_joint
%Position
figure;
for i=1:6
    subplot(3,2,i);

```

```
        plot(t,q(i,:),u'LineWidth',1.5);
        title(['Joint_',num2str(i),'_Position']);
        xlabel('Time(s)');
        ylabel('Position(rad)');
        grid on;
    end
    sgtitle('Position Profiles for All Joints');

%_Velocity
figure;
for i=1:6
    subplot(3,2,i);
    plot(t,qd(i,:),u'LineWidth',1.5);
    title(['Joint_',num2str(i),'_Velocity']);
    xlabel('Time(s)');
    ylabel(
```

References

- [1] ABB, “ABB launches its most compact and lightweight 6-axis robot ever,” Press Release, Shanghai, China, Sep. 19, 2018.
- [2] ABB, *Product specification: IRB 1100*, Document ID: 3HAC064993-001, Rev. S.
- [3] ABB, *Product manual - IRB 1100*, Document ID: 3HAC064992-001, Rev. T.